MAISON DE LA FORMATION JACQUELINE AURIOL
Génie mécanique et productique pour l'aéronautique & le spatial

UNIVERSITÉ ÉVRY PARIS-SACLAY
UFR Sciences et Technologies

Université Évry Paris-Saclay

Mention : Ingénierie des Systèmes Complexes

Parcours : M2 Robotique Industrielle

Master M2 Robotique Industrielle FI

Année universitaire

2025–2026

From a Multi-Robot Digital Twin to Safety-Gated Neuro-Symbolic Language-to-Motion Control

Université de Toulouse

Maison de la Formation Jacqueline Auriol (MFJA)

NOM :	IBRAHIM
Prénom :	Ali
Numéro d'étudiant :	20255709
Statut :	Formation initiale
Tuteurs de stage :	Diane BURY et Olivier STASSE

Identification Page

Student

Name	Ali IBRAHIM
Programme	M2 ISC — Industrial Robotics track
Academic year	2025–2026
Student number	20255709
Telephone	07 50 79 35 89
Email	ali.ibrahim@etud.univ-evry.fr

Host organisation

Organisation	Université de Toulouse
Host service	Maison de la Formation Jacqueline Auriol (MFJA)
Legal address	118 route de Narbonne, 31062 Toulouse Cedex 9, France
Internship site	1 rue Tarfaya, 31400 Toulouse, France
Telephone	05 62 25 88 80

Internship supervisors

Diane BURY	Internship supervisor — MFJA; diane.bury@utoulouse.fr Telephone: 05 62 25 88 18
Olivier STASSE	Internship supervisor — LAAS-CNRS; olivier.stasse@laas.fr Telephone: 05 61 33 79 82

Academic supervisor

Name	Nicolas SEGUY
Role	Academic supervisor
Email	nicolas.seguy@univ-evry.fr
Telephone	01 69 47 75 51 (IBISC secretariat)

Mission

Period	2 March 2026 – 28 August 2026
Host unit	MFJA
Internship title	Développement d'une interface unifiée pour robots industriels et mobiles en simulation
Report language	English

Abstract

The internship developed a common Gazebo environment for the industrial and mobile robots used at the Maison de la Formation Jacqueline Auriol (MFJA). The mission centred on a digital twin of Room 315, an industrial robotics practice site comprising two rail-transport cells and four industrial-robot workstations, together with unified Robot Operating System 2 (ROS 2) access, calibrated multi-shuttle transport and an investigation of overhead-camera perception with English task requests. The implemented solution is a schema-constrained neuro-symbolic interface: learned models propose fields in fixed task and scene formats, while deterministic checks, symbolic planning and software supervision determine which commands may execute. It is therefore not an end-to-end vision–language–action policy that maps images and language directly to robot actions.

Configuration-driven launch and typed ROS 2 interfaces connect the simulated robots, rail topology and software execution supervisor. A local Qwen2.5-1.5B-Instruct model proposes structured task fields. The paired-view ResNet-18 estimator, identified as V4 in the versioned artefacts, uses shared low-level processing and a separate higher-level output branch for each rail. A visual observation is accepted only when every present shuttle passes the identity, freshness, vocabulary, numeric and confidence checks, and task fields must belong to the permitted Room 315 domain. PlanSys2/POPF then produces a symbolic plan; the software execution supervisor authorises one plan action, translated into a typed command, at a time, and the execution layer checks its observed effect.

The visual model selected at epoch 11 was frozen and then evaluated once on a preregistered Final Test whose protocol was fixed in advance and which was separate from training, Hard-case Validation and the Development Canary. Across 1,040 scenes and 4,680 shuttle targets labelled as present, 90.8974% received the correct canonical rail-segment label with a longitudinal error no greater than 0.05, or 5% of that segment's length; all 80 predeclared acceptance checks passed.

The closed-loop Gazebo evaluation, in which each command is followed by a new observation and effect check, completed all 12 isolated runs. All 24 expected step effects and 48 software-supervisor decisions were accepted, and every shuttle controller ended in a stopped state. A separate ten-case language evaluation matched every declared outcome. The five-scenario fault-injection campaign passed 5/5 without incorrectly declaring success and left all controllers disabled. The reported results apply only to the explicitly authorised Gazebo-only runtime configuration. Command execution starts disabled and requires operator activation; once enabled and given a confirmed task, closed-loop planning and execution proceed automatically in Gazebo. The evidence therefore supports only this simulated profile; physical transfer would require additional validation, as discussed with the wider limitations and future work in Chapter 12.

Keywords: ROS 2; Gazebo Harmonic; industrial robotics; digital twin; multi-robot systems; flexible manufacturing; PDDL; PlanSys2; computer vision; language-and-vision interface; neuro-symbolic control; safety supervision.

Résumé

Le stage a produit un environnement Gazebo commun pour les robots industriels et mobiles de la Maison de la Formation Jacqueline Auriol (MFJA). La mission a réuni un jumeau numérique de la salle 315, site de travaux pratiques en robotique industrielle comprenant deux cellules de transport sur rail et quatre postes de robot industriel, un accès Robot Operating System 2 (ROS 2) unifié, un transport multi-navette calibré et une étude de perception par caméras en hauteur associée à des requêtes en anglais. La solution est une interface neuro-symbolique contrainte par schéma : les modèles appris proposent des champs dans des formats fixes de tâche et d'état de scène, tandis que les vérifications déterministes, la planification symbolique et la supervision logicielle déterminent quelles commandes peuvent être exécutées. Elle n'est donc pas une politique vision–langage–action de bout en bout qui transforme directement images et langage en actions robotiques.

Le lancement piloté par configuration et les interfaces ROS 2 typées relient les robots simulés, la topologie ferroviaire et le superviseur logiciel d'exécution. Un modèle Qwen2.5-1.5B-Instruct local propose les champs structurés de la tâche. Le ResNet-18 à deux vues partage le traitement visuel générique, puis utilise une branche de sortie de haut niveau propre à chaque rail. Une observation visuelle n'est acceptée que si chaque navette présente satisfait les contrôles d'identité, de fraîcheur, de vocabulaire, de validité numérique et de confiance ; les champs de tâche doivent aussi appartenir au domaine autorisé de la salle 315. PlanSys2/POPF produit ensuite le plan symbolique ; le superviseur logiciel d'exécution autorise une action du plan, traduite en commande typée, à la fois, puis la couche d'exécution vérifie son effet observé.

Le modèle retenu à l'issue de l'époque 11 a été figé avant d'être évalué une seule fois sur un test final défini à l'avance, distinct de l'entraînement, de la validation des cas difficiles et de la partition sentinelle de développement (Development Canary). Sur 1 040 scènes et 4 680 cibles correspondant aux navettes annotées comme présentes, 90,8974 % ont reçu l'étiquette canonique correcte de segment ferroviaire avec une erreur longitudinale au plus égale à 0,05, soit 5 % de la longueur du segment. Les 80 critères d'acceptation, fixés avant l'évaluation, ont été satisfaits.

Les 12 exécutions isolées de la campagne Gazebo en boucle fermée, où chaque commande est suivie d'une nouvelle observation et d'un contrôle de son effet, ont réussi. Les 24 effets attendus des étapes et les 48 décisions du superviseur logiciel ont été validés, et tous les contrôleurs de navette se trouvaient à l'arrêt. L'évaluation linguistique menée sur dix cas a produit les résultats attendus. La campagne d'injection de fautes a réussi ses cinq scénarios sans aucun succès déclaré à tort et a laissé tous les contrôleurs désactivés. Ces résultats concernent uniquement la configuration d'exécution réservée à Gazebo et explicitement autorisée pour cette évaluation. L'émission de commandes est désactivée au démarrage et doit être activée explicitement par l'opérateur ; une fois l'émission de commandes activée et la tâche confirmée, la planification et l'exécution en boucle fermée fonctionnent automatiquement dans Gazebo. Ces éléments n'autorisent pas un déploiement physique, qui exigerait les validations supplémentaires discutées au Chapitre 12.

Mots-clés : ROS 2 ; Gazebo ; robotique industrielle ; jumeau numérique ; système multi-robot ; PDDL ; PlanSys2 ; vision ; VLA ; commande neuro-symbolique ; supervision de sécurité.

Acknowledgements

I would like to sincerely thank Olivier Stasse and Diane Bury for their scientific guidance throughout this internship. Our weekly discussions and their constructive feedback helped me refine the technical architecture, improve the experimental methodology and develop the initial modelling assignment into a broader integration and research project for the MFJA platforms.

I would also like to thank Nicolas Seguy for his academic supervision and for following the progress of the placement. Finally, I am grateful to the administrative and technical staff at the MFJA for coordinating access to the facilities, helping with organisational procedures and providing the practical support needed throughout the internship.

Use of Generative Artificial Intelligence

Generative artificial intelligence tools, primarily OpenAI ChatGPT, were used during the preparation of this work as supporting tools. Their use included improving the grammatical correctness, clarity, and fluency of the report; enhancing the visual quality and presentation of selected figures and diagrams; and assisting with the generation, organisation, and refactoring of selected portions of code, as well as with repetitive development tasks and workflow optimisation.

All technical choices, system design, implementation, experiments, results, analyses, and conclusions presented in this report remain the author's own work and responsibility. AI-assisted code, figures, and suggestions were systematically reviewed, adapted where necessary, tested or validated as appropriate, and integrated by the author. Generative AI was therefore used as a supporting tool to improve efficiency and presentation quality, rather than as a substitute for the author's technical reasoning, engineering decisions, implementation, or validation. This disclosure follows the university reporting guidance [21].

Glossary and Acronyms

Acronyms

AI	Artificial Intelligence
APE	Activité Principale Exercée, the French principal-activity code
API	Application Programming Interface
CAD	Computer-Aided Design
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DZI	Project device-name prefix for an indexing-zone detector
F1	Harmonic mean of precision and recall
FI	Formation initiale, the non-apprenticeship study route used here
GUI	Graphical User Interface
GPU	Graphics Processing Unit
INSA	Institut National des Sciences Appliquées
INSEE	Institut National de la Statistique et des Études Économiques
ISAE	Institut Supérieur de l'Aéronautique et de l'Espace
ISC	Ingénierie des Systèmes Complexes
IoU	Intersection over Union, the overlap between predicted and reference regions divided by their union
JSON	JavaScript Object Notation
LAAS-CNRS	Laboratoire d'Analyse et d'Architecture des Systèmes, a laboratory of the Centre National de la Recherche Scientifique
LLM	Large Language Model
M2	Second year of a Master's degree
MAE	Mean Absolute Error, the average absolute difference between a prediction and its reference value
MFJA	Maison de la Formation Jacqueline Auriol
ML	Machine Learning
NAF	Nomenclature d'Activités Française
PDDL	Planning Domain Definition Language
POPF	Partial Order Planning Forwards
QoS	Quality of Service
RGB	Red–Green–Blue colour representation
ROS	Robot Operating System
SDF	Simulation Description Format
STL	Stereolithography mesh format
TF	The time-indexed ROS hierarchy of coordinate-frame transforms
URDF	Unified Robot Description Format
VLA	Vision–Language–Action

VLM	Vision–Language Model
YAML	YAML Ain't Markup Language

Project-specific terms

Acceptance gate A pass/fail criterion fixed before evaluation, distinct from a project-management decision milestone.

Approved Gazebo profile A hash-identified Gazebo runtime selected after manual review. Selection neither enables actuation nor grants physical-safety approval; operator activation is separate.

Checkpoint Model weights saved after an epoch; freezing prevents further updates before evaluation.

Closed loop Execution in which each command is followed by a new observation and a check of its expected effect before planning continues.

Digital twin A versioned digital representation of a defined physical system that includes its relevant information flows.

Fail-closed A policy that blocks an operation when required state is missing, stale, contradictory or from an unapproved source.

Hard-case Validation The current development partition used for checkpoint selection and calibration, distinct from the historical Grouped Validation and the Final Test.

Development Canary A post-selection regression partition used to check the frozen candidate without checkpoint selection or refitting.

Historical Grouped Validation The predecessor validation partition that selected the checkpoint later used to initialise the V4 backbone; it is not a direct V4 input.

Consumed historical Test The predecessor test partition that was evaluated once and is therefore no longer locked or used as the current Final Test.

Final Test The preregistered post-freeze partition used as the project's final visual-model evidence.

Neuro-symbolic interface An architecture in which learned components propose fixed-schema task and scene fields, while deterministic validation, symbolic planning and the software execution supervisor determine which commands may be published.

`ObservedState` A timestamped state snapshot whose accepted facts carry source, confidence and known/unknown/stale/conflicting status.

Public topology The common logical route graph and its canonical segment labels exposed to ROS 2 and planning, distinct from each rail's internal geometric curve identifiers.

SHA-256 A cryptographic digest used here to identify the exact files or model weights associated with an evaluation.

Shuttle An independently controlled pallet carrier that travels around one of the two Room 315 rail cells and may be empty or carry a payload.

Software execution supervisor The deterministic component that checks state/safety before publishing a typed command, distinct from the human supervisors and operator.

`TaskGoal` The immutable structured task created after a fixed-schema text candidate passes rule validation and operator confirmation.

Contents

List of Figures

List of Tables

Chapter 1

General Introduction

Context and engineering objective. The MFJA robotics platforms bring together industrial manipulators, a mobile robot, transport cells, cameras and programmable devices. Integrating this equipment in simulation requires compatible models, coordinate frames, namespaces, commands and state descriptions. The work went beyond constructing a visually accurate Gazebo scene: the simulated equipment also had to behave consistently enough to support routing, planning and experimentation.

The project also examined whether camera observations and English requests could describe a task without sending motor commands directly. The language and vision paths supply information about the request and the scene. PlanSys2 produces the symbolic sequence, and the software execution supervisor checks each requested step before execution. Neither learned model produces a plan or an action vector.

Objectives and principal contribution. The project pursued four engineering objectives within simulation:

O1. establish a maintainable Gazebo representation of the MFJA third floor and Room 315, with configuration-driven launch and unified ROS 2 access to the supported simulated robots;

O2. implement continuous multi-shuttle transport over calibrated rail geometry and explicit topology, including device semantics, routing, reservations and collision-avoidance constraints;

O3. define typed command and state interfaces, together with supervision rules for checking commands and confirming their effects; and

O4. design and evaluate a neuro-symbolic workflow that combines natural-language goal interpretation and paired-camera state estimation with PDDL/PlanSys2 planning and software-supervised execution.

The four objectives form a progression from the simulated environment and its transport system to task-level interaction through language and vision. The objective identifiers O1–O4 are used again in table 11.5 to connect each objective to its evaluated result.

Method and headline results. Development followed short build, test and review cycles. Each part was first implemented and checked separately, then connected to the preceding layers. This made it possible to resolve modelling, interface and coordination issues before evaluating the complete workflow. The software, dataset and model versions used for the reported evaluations are listed in appendix A.

The frozen visual estimator passed all predeclared Final Test acceptance checks, while all 12 isolated English-to-motion runs reached their declared terminal state. Chapter 11 reports the detailed metrics, evidence and simulation-only scope.

Report structure. The host environment, mission and principal design choices are introduced in chapters 2 to 4. The engineering foundations are then developed in chapters 5 to 8, followed by the language integration, paired-camera data generation and visual training process in chapter 9. Chapter 10 follows one English instruction through planning, software-supervised motion and result checking, and chapters 11 and 12 present the results and future work.

Chapter 2

Host Organisation and MFJA Robotics Platforms

2.1 Université de Toulouse

The internship was formally hosted by Université de Toulouse, whose legal address is 118 route de Narbonne, 31062 Toulouse Cedex 9. The work itself was carried out at the MFJA site, 1 rue Tarfaya, 31400 Toulouse. As a public university, the institution combines higher education, research and knowledge transfer. Its official Nomenclature d'activités française/Activité principale exercée (NAF/APE) code is 85.42Z [8], which the French national statistics institute (INSEE) classifies as *higher education*, including first-, second- and third-cycle university programmes [7].

Université de Toulouse has operated as an experimental public institution since January 2025. Official figures published for 2026 indicate approximately 39,000 students and 4,200 staff [20]. These figures concern the university as a whole, while the internship took place within the smaller MFJA environment.

2.2 Maison de la Formation Jacqueline Auriol

The MFJA is a shared centre for engineering education on the Toulouse Aerospace innovation campus. Its activities cover research and innovation, knowledge and skills transfer, and initial and continuing education. According to its official presentation, the site brings together more than 1,500 students from 15 mechanical-engineering programmes associated with Université de Toulouse, INSA Toulouse and ISAE-SUPAERO. It supports work in mechanical engineering, manufacturing, aeronautics, automation and robotics in collaboration with industrial partners and research laboratories [19].

Because the facilities are shared across teaching and research activities, the software developed for the platform must support several configurations, remain usable by different operators and be maintainable beyond a single development environment. These requirements influenced the package and launch architecture presented in Chapters 5–6.

2.3 The Room 315 robotics environment

Room 315 contains two flexible transport cells: closed-loop rail systems whose independently controlled pallet carriers are termed *shuttles*. The cells share one public topology—the canonical segment names and connectivity exposed to ROS 2 and planning—but have distinct calibrated geometries, meaning that each rail follows its own measured spatial path. Each cell has four numbered shuttle slots. Four industrial-robot workstations surround the two cells. In the configuration used for this project, right-rail slots 1–2 are served by the Yaskawa HC10DT workstation and slots 3–4 by the Stäubli TX2-60L workstation; left-rail slots 1–2 are served by the Yaskawa HC10 workstation and slots 3–4 by the KUKA KR6 R900 sixx workstation. Each circuit contains outer and inner branches, four switching zones, indexing detectors and stoppers. A switching zone diverts a shuttle between branches, an indexing detector reports its alignment or presence, and a stopper holds or releases it. A shuttle may be empty or carry a payload.

This arrangement brings together several engineering disciplines:

- mechanical layout, computer-aided design (CAD), kinematics and collision geometry;
- industrial automation, sensors, stoppers and route switching;

- robot middleware, namespace prefixes that isolate robot interfaces, timing and distributed communication;
- planning, scheduling and resource conflicts among shuttles sharing rail sections;
- computer vision, data collection and model evaluation;
- human–robot interaction through task-level language.

Together, these disciplines make Room 315 a practical platform for studying the integration of robotics, transport, perception and control. Figure 2.1 shows the simulated layout; Chapters 7–8 define the rail topology, device semantics and supervisory constraints.

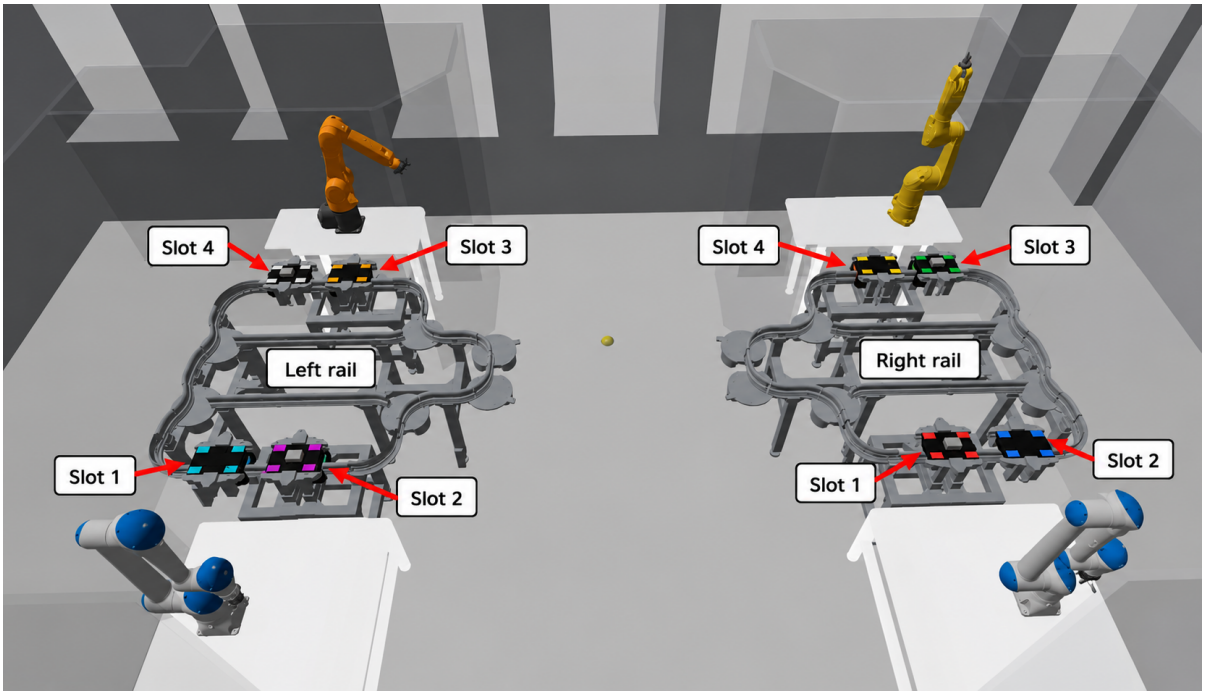


Figure 2.1: Gazebo view of Room 315. Four industrial-robot workstations surround two rail-transport cells containing eight stopped shuttles. The image identifies Slots 1–4 on the left and right rails. The measured rail geometries and device locations are presented in chapter 7.

2.4 Organisation and responsibilities

Figure 2.2 presents the supervision and responsibilities directly associated with the internship.

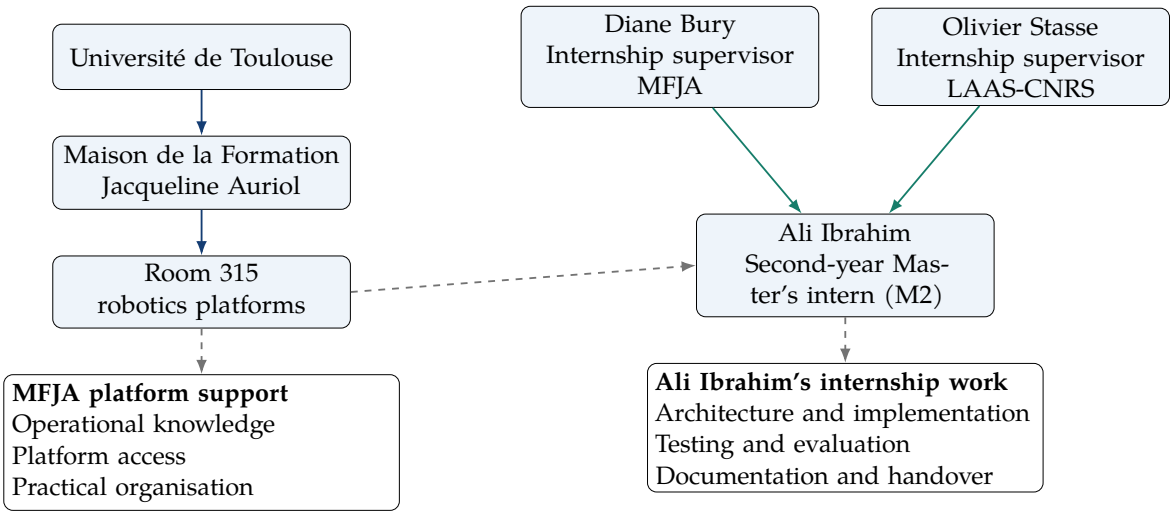


Figure 2.2: Host-unit placement, parallel supervision and distribution of internship responsibilities.

Diane Bury and Olivier Stasse were the two internship supervisors, with equal supervisory roles. Diane Bury was based at MFJA and Olivier Stasse at LAAS-CNRS. Together they provided scientific direction and reviewed choices related to the platform. MFJA personnel supported access, scheduling and practical organisation. I was responsible for the architecture, implementation, evaluation, documentation and handover described in chapter 3.

Within the MFJA, the host unit for this work was the Room 315 robotics-platform activity, connecting shared teaching infrastructure with research prototyping.

2.5 Position of the mission within the host

The mission connects platform operation with research prototyping. Platform managers, students and researchers can use the system to launch robots and rail scenarios reproducibly. The same foundation can support synthetic-data generation, task-planning experiments and comparisons between simulation configurations. Possible uses include a teaching exercise on ROS 2 topics, a planner benchmark, a visual-perception dataset or a simulation demonstrator.

This range of uses motivated the emphasis on reusable configuration, explicit interfaces and documented operating procedures.

2.6 Economic and strategic value

Because the host is a public university rather than a commercial company, turnover and export revenue are not meaningful indicators of this mission's value. To indicate the scale of the institution, the university's 2026–2029 strategic document reports a 2024 global budget of nearly EUR 474 million and EUR 114 million in purchases [20]. These university-wide figures provide context, not a financial return for this project. The value of the mission lies instead in practical reuse, training capacity and reduced integration effort. The main benefits are:

- **reuse**: one model and launch framework avoids recreating an integration for each practical class or research project;
- **risk reduction**: simulation and supervised execution check naming, topology and commands before access to costly equipment;
- **asset visibility**: a digital twin makes robot capabilities available for remote preparation, including capabilities that might otherwise remain under-used;
- **training capacity**: repeatable scenario generation creates data and exercises without monopolising the physical cell;
- **maintenance**: typed interfaces, tests and documentation reduce dependence on knowledge held by a single developer;
- **technology transfer**: the MFJA can demonstrate current ROS 2, planning and AI methods while maintaining clear safety boundaries.

These benefits align with the MFJA's institutional role of sharing costly engineering resources and connecting education with contemporary industrial methods.

Chapter 3

Mission, Project Management and Personal Contribution

3.1 Mission and work packages

The internship title used in this report is *Développement d'une interface unifiée pour robots industriels et mobiles en simulation*. The signed agreement identifies the placement, host, dates and entrusted activities [22]. The work reported here concerns simulation; no deployment on physical platforms is claimed. The first practical assignment was to model Room 315 in Gazebo and establish a usable simulation environment for its robotic and transport equipment. Completing this objective ahead of schedule left time for a research work package combining overhead-camera perception with natural-language task interaction. The internship dates were unchanged.

The implemented work is organised into two phases:

Phase 1—simulation foundation Room 315 Gazebo modelling and a reusable, selectively launched multi-robot and rail simulation foundation.

Phase 2—neuro-symbolic research Paired cameras, visual-state estimation, validated language goals, PDDL/PlanSys2 and safety-gated neuro-symbolic control, in which learned proposals are checked before symbolic planning and software-supervised execution.

The phases describe the project chronology, whereas objectives O1–O4 in chapter 1 define the units used to assess the completed work. The simulation phase established the multi-robot and transport foundations. The neuro-symbolic phase extended the typed interfaces and connected visual and language inputs to planning, software-supervised execution and effect verification.

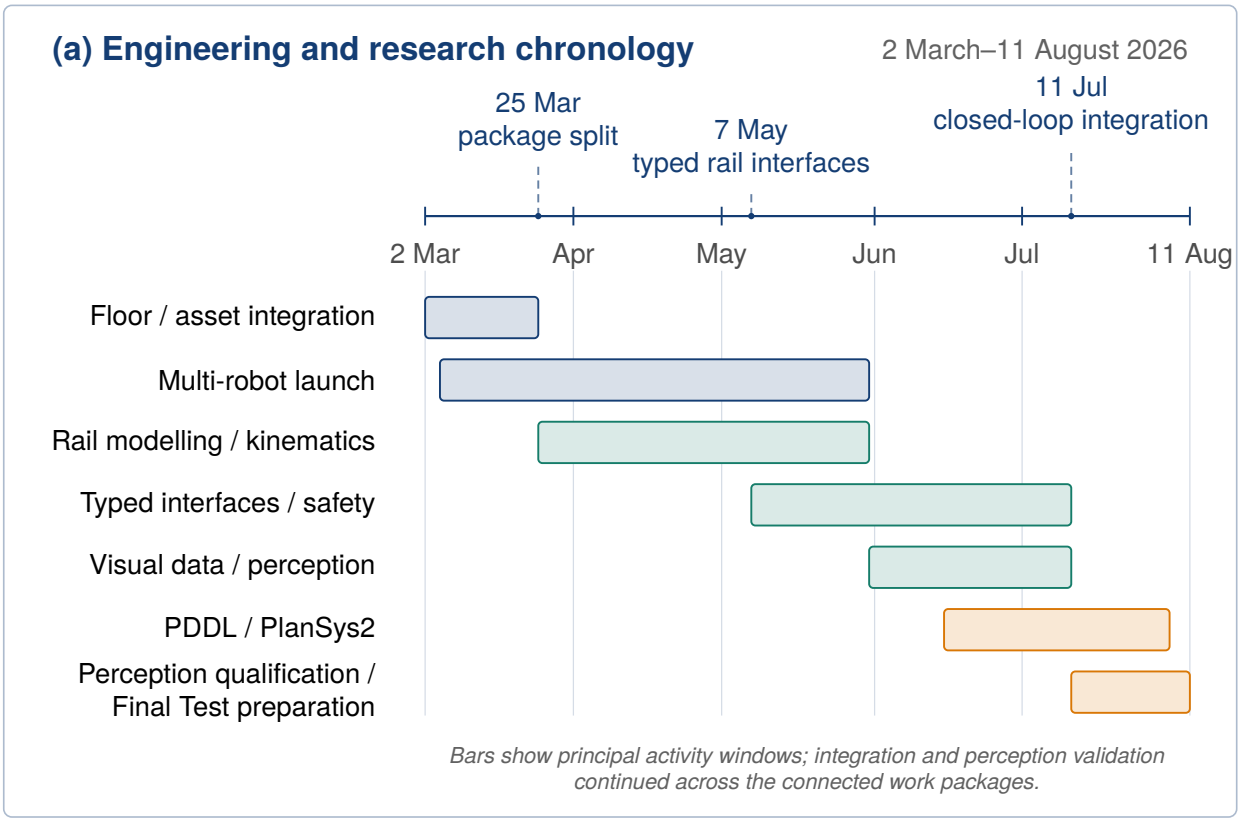
3.2 Chronology and decision milestones

The internship schedule was fixed from the outset. As shown in figure 3.1, the technical phases overlap because integration, testing and correction continued after each main decision milestone.

3.3 Decision progression

Figure 3.2 summarises how the project progressed from the initial simulation to the integrated neuro-symbolic system. Each stage depended on working results from the preceding stage.

Major technical decisions were taken after weekly discussions with the supervisors. The agreed solutions were then implemented and checked through configuration, tests and integration runs.



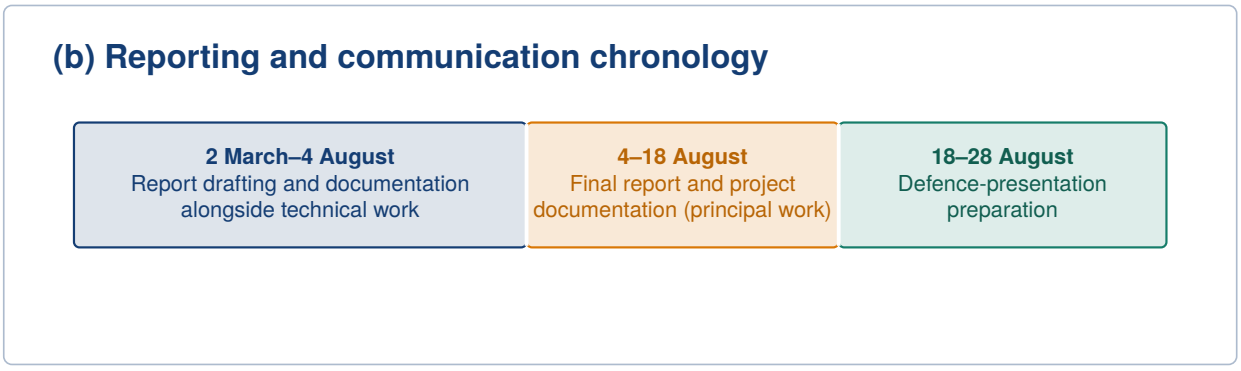


Figure 3.1: Internship chronology as of 11 August 2026. Engineering, integration, visual-model validation and the preregistered Final Test extend through 11 August in panel (a), overlapping the report-focused period that began on 4 August. Documentation continues to 18 August; the final period, 18–28 August, is reserved for defence-presentation preparation.

Phase 1 — initial assigned task

1. World and robots

Engineering need: unify heterogeneous assets, names and topics. **Implementation:** correct the CAD scene, separate package ownership and launch from a configuration registry that assigns each robot a unique namespace. **Result:** select any supported simulated-robot subset.

↓

2. Shuttle motion

Engineering need: repeatable motion through switch transitions. **Implementation:** use arc-length Hermite paths and Gazebo pose control. **Result:** repeatable continuous motion with an explicit kinematic model.

↓

3. Topology and devices

Engineering need: encode legal routes, stops and device semantics. **Implementation:** declare directed segments, switches, slots, stoppers and sensors in YAML. **Result:** planner-readable dual-rail state and coordination.

Phase 2 — additional research direction

4. Visual and task interfaces

Engineering need: interpret English tasks and the current camera scene. **Implementation:** pair overhead RGB views, keep labels separate from model input, and convert fixed-schema task-field proposals into confirmed task goals. **Result:** structured language and visual inputs for planning.

↓

5. Planning and software-supervised execution

Engineering need: convert the current task and state into software-supervised motion. **Implementation:** rebuild the PDDL problem from the latest accepted state, select one symbolic step, translate it, pass it through the software execution supervisor and check the resulting state. **Result:** one-step planning and execution that repeats until completion.

↓

6. Experiments and results

Engineering need: measure perception and confirm complete task behaviour. **Implementation:** use declared visual partitions, model metrics, software tests and a 12-run clean-launch operator-interface campaign. **Result:** model measurements and complete positive runs across both rails, all eight identities and five case families.

Figure 3.2: Project progression as a dependency chain. Each stage converts a technical objective into an engineering result required by the next stage.

3.4 Requirements and review process

Requirements were expressed at three levels. Launch arguments, topics and runbooks define how operators use the system. Executable contracts and tests specify what each component must accept, reject or preserve. Artefact manifests and hashes give datasets and models reproducible identities. The main requirements across these levels were robot isolation without unintended cross-namespace communication, stable shuttle identity across interfaces, partial device updates without unintended changes, current state before planning, deterministic translation from each symbolic plan step to a typed command and a post-command observation after each executed plan step.

Each property was checked where it was implemented: geometry and coordinate frames, interface semantics, shared-rail coordination and learned-model output. The safeguards and data-leakage controls are detailed in chapters 7 to 9.

The agreed solutions were reflected in configuration and version-controlled code, then exercised through the corresponding tests and experiments. The identifiers for the relevant sources, datasets and models are listed in appendix A; the corresponding objective-to-result assessment is reported in table 11.5.

3.5 Personal responsibility, collaboration and resources

Within the shared repository, I was responsible for the third-floor and Room 315 models; the selective multi-robot launch architecture; calibrated rail transport, device logic and typed interfaces; task contracts, PDDL/PlanSys2 integration and software-supervised one-step execution; the paired-camera dataset and visual training pipeline; and the tests, manifests, runbooks and handover documentation.

The supervisors provided scientific direction, technical review and knowledge of the platform. During the weekly discussions, I presented the alternatives, the available data and the consequences for shared interfaces, software safeguards and experimental claims. After we agreed on a decision, I implemented it in the relevant configuration, code, tests and documentation.

In practice, resource management concerned the fixed schedule, shared platform access, simulation and training time, dataset and checkpoint storage, and the testing workload. I kept compute-intensive perception and language nodes optional, prioritised reusable configuration and regression tests, and prepared build and run instructions so that the host could continue using the work. No purchasing budget was assigned to me. The resources I had to manage were GPU time, storage capacity, simulation time and access to the shared platform.

3.6 Autonomy and reflective engineering examples

I divided the broad integration objective into testable increments and kept early implementation choices reversible. Before keeping a design, I checked that its interfaces and implementation were consistent and that the corresponding regression tests passed. I discussed decisions affecting shared interfaces, experimental claims or software safeguards with the supervisors before finalising them. The following two examples show how this worked in practice.

3.6.1 Selecting a defensible level of rail fidelity

At the beginning of the rail work, I compared a dynamic wheel–rail simulation with a kinematic rail-following model. The measured mechanical parameters required for a credible dynamic model were not available, and that option was expected to require more integration time and computation. The project questions instead concerned routing, sensor crossings, slot arrival, perception and supervised execution. I therefore proposed the calibrated graph and arc-length solution detailed in

chapter 7. I discussed the comparison with the supervisors during a weekly review. The agreed priority was repeatable behaviour at the level required by those questions. Mechanical effects that could not be supported by the available data were left outside the model. I then implemented the model, checked the configured geometry and tested routing, sensor feedback and slot-arrival behaviour. This experience showed me that model fidelity had to follow the question being studied. In this case, the missing mechanical data, the schedule and the available computing capacity were practical design constraints.

3.6.2 Choosing PDDL/PlanSys2 over end-to-end VLA control

For the second phase, I compared an end-to-end vision–language–action model that would infer actions directly with VLA and hybrid planning architectures discussed with my supervisors. This comparison supported a clear division of responsibilities: the visual model estimates the scene state from the paired camera images, while PDDL/PlanSys2 determines the action sequence, whose actions are then authorised by the software execution supervisor and applied by the controllers. This approach was better suited to Room 315 because its rail topology and device rules are explicit, whereas the available dataset contains scene-state labels rather than action demonstrations. It also made the plans easier to inspect and execution errors easier to locate. This became the architecture implemented in chapters 9 and 10.

3.7 Competencies and professional learning

Table 3.1 relates the principal competencies developed during the internship to concrete examples from the project.

Table 3.1: Engineering competencies and concrete project examples.

Competency	**Project example**
Systems analysis	Separation of geometry, perception, planning, supervision and exclusive command-publication authority
Robotics integration	ROS 2/Gazebo assets, controllers, launch composition, namespaces and device feedback
Algorithm design	Arc-length graph, topology-aware routing, reservations, clearance and effect checks
AI/ML method	Local language-model integration, leakage-controlled paired-image data, declared partitions and separate visual-output heads
Software quality	Typed contracts, focused regression tests, configuration manifests and reproducible result records
Project management and communication	Incremental milestones, supervisory reviews, diagrams, runbooks, result tables and explicit scope

These two decisions changed how I compared technical alternatives. I used three questions throughout the later integration work: what data were available, what behaviour Room 315 required, and how the result could be tested.

Chapter 4

Technical Background and Design Choices

4.1 ROS 2 as the integration layer

ROS 2 provides processes (nodes), typed interfaces, discovery and distributed communication. Topics suit continuous streams, services suit short request–response operations and actions suit long-running tasks with feedback [15]. These distinctions guided the choice of typed streams, explicit request–response operations and isolated namespaces for the project. Here, a typed interface has a fixed message schema, discovery allows ROS 2 processes to find one another, and a namespace is a name prefix that prevents one robot's nodes and topics from colliding with another's. The resulting interfaces are presented in chapters 6 and 8.

ROS 2 Quality of Service (QoS) policies configure delivery properties such as reliability and deadline; the standard sensor-data profile explicitly favours timely receipt over guaranteed delivery of every sample [16]. In this project, camera data, state and commands have different reliability and latency needs. The design therefore combines conservative communication defaults with explicit freshness checks. Profiles for other network conditions would require measurement in their target environment.

4.2 Gazebo Harmonic and description formats

Gazebo Harmonic is the recommended Gazebo pairing for ROS 2 Jazzy on Ubuntu 24.04 [13]. At its root, the Simulation Description Format (SDF) can contain a model, actor, light or one or more worlds; a world encapsulates models, scene, physics and plugins. SDF also defines sensor elements on links or joints, including their type and properties [17]. The Unified Robot Description Format (URDF) remains useful for robot kinematic trees and `robot_state_publisher`. The `ros_gz_bridge` translates selected messages between ROS 2 and Gazebo Transport, Gazebo's native messaging layer [14].

Simulation serves four complementary purposes in this project:

1. validate coordinate frames, namespaces and controller interfaces across heterogeneous robot models in a controlled environment;
2. support instruction in multi-robot launch and command workflows;
3. execute repeatable rail and safety scenarios at scale;
4. generate labelled overhead observations for perception training.

These purposes impose different fidelity requirements. High-resolution visual meshes support camera-based observation, while simplified collision geometry keeps contact handling efficient and predictable. The shuttle study requires continuous, repeatable transport rather than detailed wheel–rail contact dynamics; it therefore uses kinematic pose control, in which the controller prescribes a shuttle pose along the calibrated path instead of simulating wheel–rail forces. The corresponding transport assumptions are detailed in chapter 7.

ISO 23247-1 provides an overview and general principles for a manufacturing digital-twin framework, including terms, definitions and framework requirements [9]. In this project, the digital-twin model combines a versioned representation of the simulated environment with clearly defined information flows, assumptions and boundaries. Visual similarity alone is insufficient for that purpose.

4.3 Coordinate and rail representations

A rail path is represented by piecewise geometric segments. Each segment has an arc-length coordinate s, a physical length L, and a normalised coordinate

$$\rho = \frac{s}{L}, \qquad 0 \le \rho \le 1. \tag{4.1}$$

Using equation (4.1), the controller evaluates position and tangent from s, then constructs the shuttle pose and heading. Cubic Hermite interpolation provides continuous tangents between the ordered CAD/CSV centre-line samples after their configured mapping into Gazebo. A separate topology graph defines which segments connect for each switch assignment. Geometry therefore determines *where the rail is*, while topology determines *which route is open*. This relationship is shown in figure 4.1.

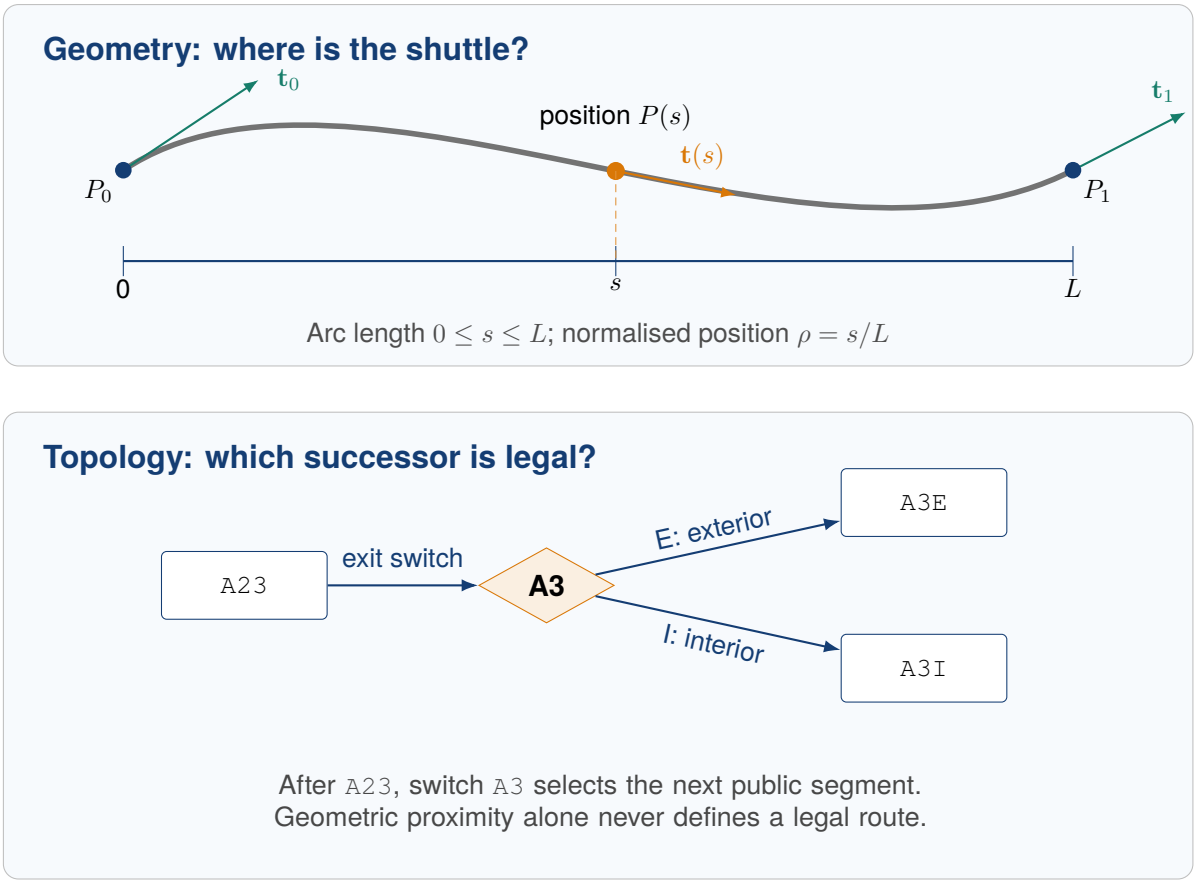


Figure 4.1: Complementary rail representations. Hermite geometry supplies position and tangent for pose construction; topology and switch state select the valid successor exposed through the public interface.

The normalised coordinate supports relative placement, while s preserves the metric distance needed for motion control. Chapter 7 presents the transport implementation, and chapters 8 to 10 define the visual-state contract and final arrival checks.

4.4 Symbolic task planning

The Planning Domain Definition Language (PDDL) describes objects, predicates, actions, preconditions and effects [12]. PlanSys2 integrates PDDL-based planning into ROS 2 and exposes domain, problem, planner and executor components [11]. The selected solver is POPF. Its original paper describes it as a forward-chaining partial-order planner [2].

Symbolic planning is appropriate for Room 315 because the important decisions are discrete and relational: which shuttle is present, which slot is occupied, which branch is selected, whether a stopper is open, whether a blocker must be staged and whether the goal has been achieved. Here, a blocker is another shuttle occupying the requested route, and staging moves it temporarily to a designated interior holding segment. Continuous motion remains inside the controller and effect-verification layers.

The planner provides the symbolic sequence, while continuous movement is handled by the controller. The simulated runtime combines the PlanSys2 planner service with project-specific translation, supervision and execution components. These component responsibilities are defined in chapter 10.

4.5 Architectural alternatives and final design

The term vision–language–action covers architectures with materially different responsibility splits. RT-2 maps images and language to tokenised robot actions inside one learned policy, while TinyVLA studies a smaller policy in the same end-to-end family [23, 25]. Hybrid systems separate more of the decision process: SayCan combines language-model scores with value functions associated with a predefined set of skills; those value functions ground skill feasibility in the physical environment [6]. OWL-TAMP uses a vision–language model to generate discrete action-ordering and continuous constraints, after which a task-and-motion planner produces a plan that satisfies them [10]. A recent structured long-horizon manipulation study also compares VLA policies with neuro-symbolic sequencing; its result is specific to that task setting rather than a universal ranking of the two families [3]. Table 4.1 summarises the resulting design comparison.

Table 4.1: Representative architectures compared by the roles assigned to learning, planning and command execution.

Approach	Learned output	Planning role	Execution path	Decision for Room 315
End-to-end VLA policy	Robot-action tokens or trajectories	Action selection inside the learned policy; no separate symbolic planner	Decoded action passed to the robot control stack	Not selected: the training corpus labels scene state rather than action trajectories, and the rail rules were already explicit.
VLM/LLM with skills or a planner	Ranked skills, subgoals or planning constraints	Skill selector or task-and-motion planner	Selected skill or downstream controller	Related design pattern, but not adopted as-is: open-world constraint generation was unnecessary for the fixed topology.
Neuro-symbolic planning with learned control	Symbolic abstractions and low-level continuous-control policies learned from demonstrations	A PDDL planner determines the high-level sequence	Learned controller executes each planned PDDL action	Closest architectural family; adapted because Room 315 learns task and scene inputs but retains an explicit controller.
Implemented Room 315 system	Fixed-schema task fields and a paired-camera structured state candidate that must pass validation; neither contains a plan or action	PlanSys2/POPF for normal planning; the stepwise execution component may add one explicit preparatory action that returns a shuttle to the canonical exterior route	Software execution supervisor publishes a typed command; the rail controller applies it	Implemented: matches the state-labelled corpus, fixed topology and simulation objectives.

For the rest of the report, VLA refers to the Phase 2 research direction and to learned-action systems such as RT-2. The Room 315 implementation is described as a neuro-symbolic language-and-vision interface.

4.6 Visual-state model

The perception system uses ResNet-18 as its visual feature extractor, commonly called a backbone. The original paper defines the residual architecture and evaluates its 18-layer variant [5]. The paired-view estimator applies one shared set of low-level weights through ResNet `layer3` independently to each image. Each view then has its own higher-level `layer4` and final prediction module (head). Its four fixed identity queries are each assigned to one known shuttle on that rail; there is no cross-camera feature path. This arrangement reuses generic edges, textures and shuttle appearance while allowing rail-specific late features to represent the non-symmetric geometries. Side is derived deterministically from the configured identity—L1–L4 denote the left rail and R1–R4 the right rail—so the learned outputs concentrate on segment, loaded state, bounding box and normalised position.

Here, loaded state means whether the shuttle carries a payload. The fixed-camera, fixed-eight-shuttle formulation produces the compact state required by the Room 315 demonstrator and remains specific to this room and fleet. Its inputs, outputs, presence handling and training process are specified in chapter 9. The simulated command and state checks performed by the software execution supervisor are defined in chapter 8; their engineering limits are discussed in chapter 12.

Chapter 5

Building the MFJA Digital Twin

5.1 Reconstruction strategy

The work began with a minimal MFJA third-floor model, establishing a recognisable, geometrically coherent environment before adding behaviour. The third-floor mesh was imported and reoriented, the ground plane enlarged, and Room 315 completed with doors and an interior window. Furniture, protective guards, robot tables and laboratory assets followed.

Each asset required consistent units, origin, pose, visual material and collision representation. CAD origins are often inconvenient manufacturing datums, while Stereolithography (STL) files contain no unit metadata. Explicit scale and orientation checks therefore aligned visual and collision geometry with the intended room coordinates. Integration proceeded incrementally:

1. load one asset in an isolated scene;
2. verify scale and orientation against known room dimensions;
3. establish a stable model frame;
4. separate visual and collision geometry where necessary;
5. place the asset in the full floor;
6. add a static test for paths and SDF structure.

The KUKA model followed this process, using a targeted, documented collision-mesh configuration that preserved the required collision behaviour.

5.2 World decomposition

The final description package provides three principal world configurations:

- `mfja_3rd_floor.world` for the shared third-floor environment;
- `room_315_only.world` for fast, focused rail, perception and language-integration work;
- isolated industrial-robot worlds assembled by launch for controller and geometry checks.

Because pose-update, entity-creation and removal service names include the internal Gazebo world name, that name forms part of the interface. The bring-up package explicitly passes both the selected world and its internal name, keeping shuttle-controller services aligned with the active world; the runbook checks the available services. Figure 5.1 shows the reconstruction.

5.3 Repository refactoring

As assets and launch components grew, the flat layout was refactored on 25 March into one repository containing several ROS packages. Its root ceased to be a ROS package, and four packages received separate responsibilities:

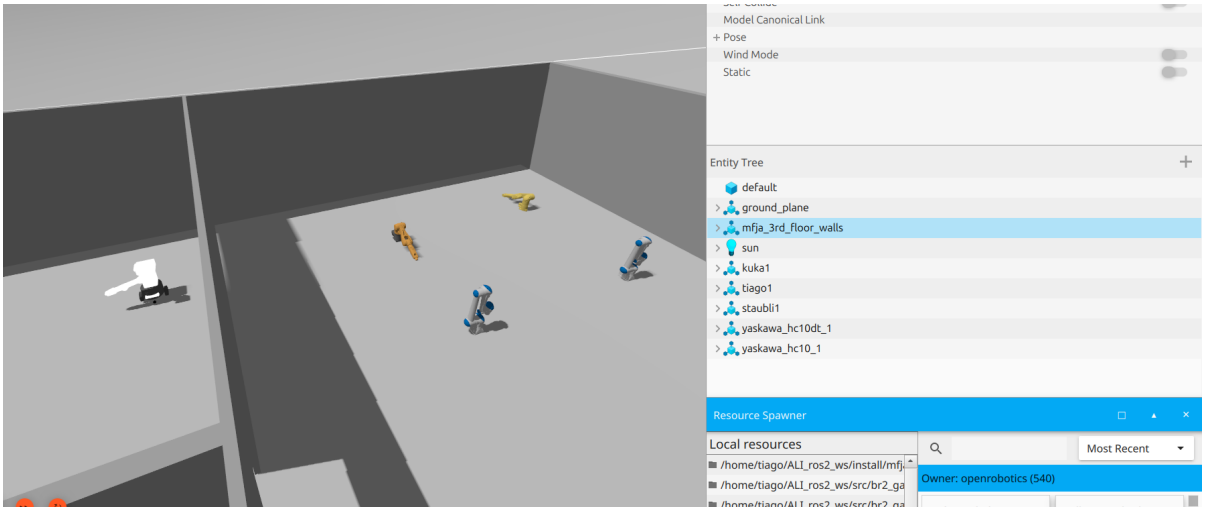
`mfja_3rd_floor_description` SDF, URDF, worlds, models, meshes and static description tests;

`mfja_3rd_floor_bringup` user-facing launch entry points and shared floor/Room 315 orchestration;

`mfja_rail_interfaces` custom messages and the shuttle-add service;

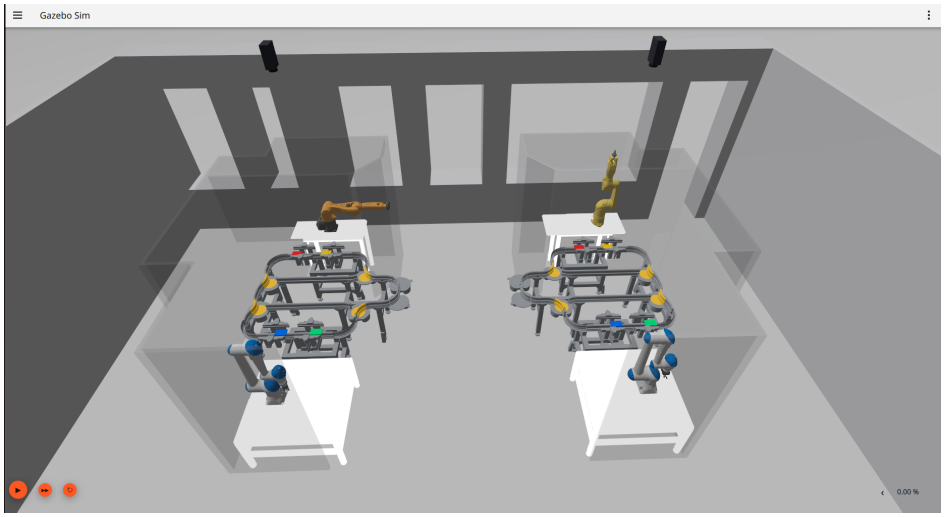
`mfja_robot_control_config` robot configuration, rail controllers, planning, perception, learning tools, launches and tests.

This gives dependencies a clear direction: description assets remain separate from planning code; interfaces remain independent of controller implementation; bring-up composes the system with-



(a) Earliest documented reconstruction stage produced during the internship.

(b) Reconstructed third-floor context.



(c) Room 315 cells and robots.

Figure 5.1: The reconstruction progressed from the earliest retained intermediate stage to the complete building context and the detailed Room 315 model.

out control algorithms; and control/configuration depends on description and interfaces. It also supports selective builds with `colcon`, the ROS 2 workspace build tool. Figure 5.2 shows the ownership and composition boundaries.

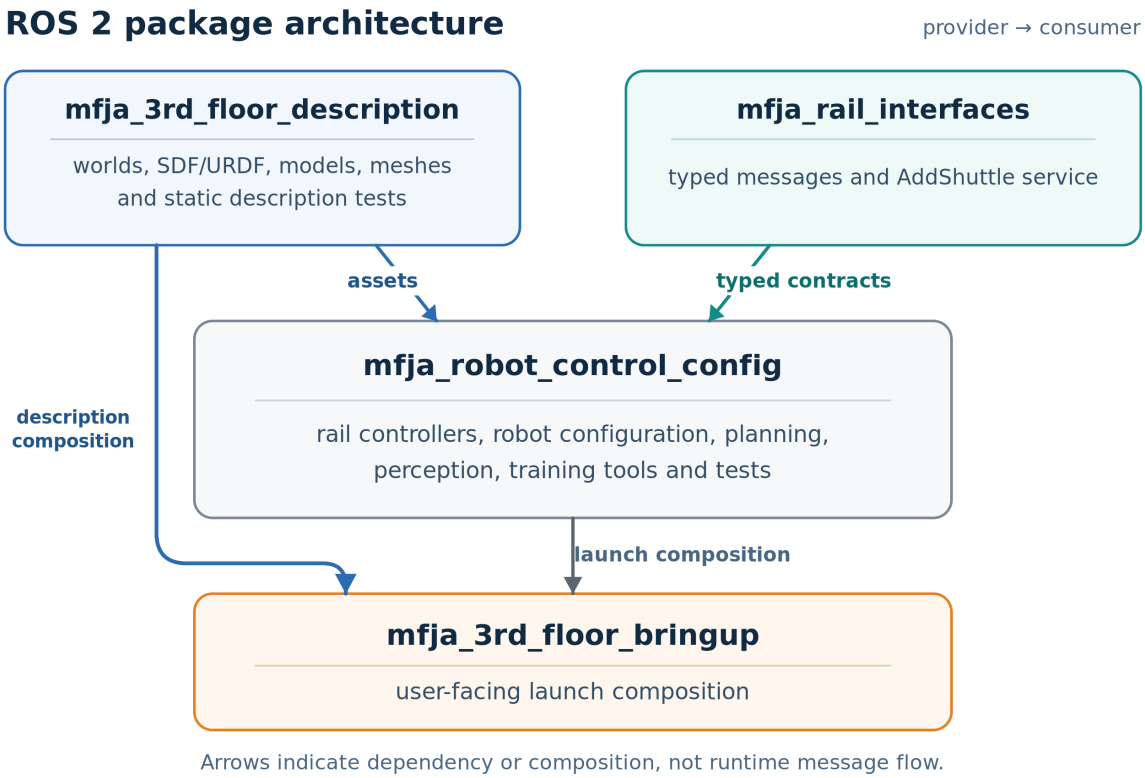


Figure 5.2: ROS 2 package ownership and composition. Description assets and typed rail contracts feed the control/configuration package; the bring-up package provides user-facing composition. Arrows represent dependency or launch composition, not runtime ROS message flow.

5.4 Model integration

5.4.1 Industrial robots

The fixed-base robots are a KUKA KR6 R900 sixx, Stäubli TX2-60L, Yaskawa HC10 and Yaskawa HC10DT. Each model includes a link–joint kinematic tree, separate visual and collision geometry, mounting geometry and a controller-facing joint list. Integration preserves vendor-specific joint structure while exposing a common higher-level command tool.

5.4.2 TIAGo

TIAGo has a mobile base, torso, arm and head, unlike the industrial arms; a base-only variant supports focused tests. Chapter 6 describes its embodiment-specific command and feedback surfaces.

5.4.3 Room and device assets

Beyond the room shell, Room 315 has separate models for the static left and right cell frames, rail paths, switches, shuttles, carried payloads, cameras and visual markers. Markers aid inspection and can be hidden for clean camera capture. The task uses two payload states: *loaded* and *empty*.

5.5 Launch architecture

The bring-up package exposes three stable entry points: `full_floor.launch.py`, `room_315_only.launch.py` and `single_industrial_robot.launch.py`. Shared Room 315 arguments are declared once in `room_315_floor_common.py`. This avoids configuration drift in which the full-floor and room-only launch files gradually develop different parameter names or defaults.

Launch choices cover graphical user interface (GUI) or headless mode (no Gazebo graphical window), pause state, selected robots, kinematic shuttles, Room 315 overhead cameras, visual markers, active shuttle identities, starting slots and the initially loaded shuttle. Defaults leave the scene inactive or readily inspectable; computationally expensive or research-specific nodes require explicit activation. The launch layer also starts the ROS–Gazebo bridges for entity creation, removal and pose updates.

Listing 5.1: Focused Room 315 launch used as the basic transport test.

```
ros2 launch mfja_3rd_floor_bringup room_315_only.launch.py \
  robots:=none \
  start_paused:=false \
  gui:=true \
  enable_room315_kinematic_shuttles:=true
```

5.6 Operational profiles

The launch layer combines installed assets into repeatable operational profiles: documented launch choices for a particular experiment. Operators can isolate an integration boundary, control computational load and scale to the shared environment without changing package ownership or public namespaces. Profiles are documented in `runbook.html` and `docs/QUICK_START_AND_FEATURE_GUIDE.md`; their verification status is presented in chapter 11.

5.7 Build and environment reproducibility

The baseline is Ubuntu 24.04, ROS 2 Jazzy and Gazebo Harmonic in a standard `colcon` workspace. Install rules cover the runtime scripts, configuration, launch files, models and interfaces. Recorded campaigns and the full-floor smoke test used executables and launch files from the built install space. A smoke test is a narrow startup and interface check, not a performance or reliability experiment. Appendix A specifies the environment and build procedure.

5.8 Outcome and model fidelity

This phase produced usable full-floor and Room 315 simulation models, selectable robots and a package structure supporting transport, perception, language interpretation and planning. Recorded evaluations include one all-robot full-floor cold start—new ROS and Gazebo processes rather than a reused session—and the Room 315 integrated campaign. Later chapters report transport, perception and result checking separately; chapter 7 details the transport model and kinematic assumptions, and chapter 12 discusses the next evaluation steps.

Chapter 6

Unified Multi-Robot Access and Control

6.1 What "unified" means

In this project, a *unified* access layer is defined by four concrete properties:

1. common YAML configuration declares selectable robot instances;
2. a common launch path spawns the selected instances and their bridges;
3. a separate common command helper validates a named robot and its joint vector;
4. instance-derived namespaces isolate commands and feedback.

The layer preserves operational differences. Fixed-base arms use joint trajectories; TIAGo also exposes base velocity and a linear torso joint. Joint limits, counts, names and units remain robot-specific. The common helper validates the selector, vector and units for `/joint_trajectory`, while structure-specific interfaces such as `/cmd_vel` remain separate.

6.2 Multi-robot operation in one Gazebo process

The first phase established per-instance command interfaces for heterogeneous robots in one Gazebo process. Each selected YAML entry defines an instance name, model, spawn pose and enabled state. Shared launch rejects duplicate names and derives namespace and bridge rules from that identity. For each supported structure, the helper has a separate command definition—aliases, joint order, topic and units—distinct from the operational launch profiles in chapter 5. Thus selection, spawning, commands and feedback share one identity chain. The retained live smoke covers the all-robot profile; automated suites in chapter 11 cover the other selector forms.

6.3 Configuration-driven robot registry

Robot instances come from YAML entries containing the instance name, model, spawn pose and enabled flag. The full floor and Room 315 may use different files with the same loader. Launch derives SDF/URDF paths, ROS namespace and bridge topics from these fields. Users may select a full name, derived short alias, numeric order, comma-separated set, `all` or `none`; table 6.1 summarises the principal surfaces.

The operator helper uses a separate validated Python table for aliases, joint ordering, command topics and angular or linear unit semantics. Tests cover both configuration boundaries, although YAML entries and helper definitions are not generated from one source.

YAML-driven launch eliminated duplicated launch scripts for every robot combination. Tests check unique instance names and asset resolution; helper tests check definition selection, joint counts, units and command topics.

6.4 Dynamic bridge generation

Gazebo Transport and ROS 2 use different communication middleware and message types. Launch derives each selected robot's bridge configuration and starts `ros_gz_bridge` instances. Joint commands travel from ROS 2 to Gazebo; joint state, odometry and transform information return where required. Under the official bridge model, YAML connects a ROS topic, Gazebo topic, corresponding message types and communication direction [14].

Table 6.1: Principal robot selectors and command surfaces.

Selector	Instance	Type	Primary command
`kuka`	`kuka1`	KUKA KR6	`/kuka1/ joint_trajectory`
`staubli`	`staubli1`	Stäubli TX2	`/staubli1/ joint_trajectory`
`hc10`	`yaskawa_hc10_1`	Yaskawa HC10	`/yaskawa_hc10_1/ joint_trajectory`
`hc10dt`	`yaskawa_hc10dt_1`	Yaskawa HC10DT	`/yaskawa_hc10dt_1/ joint_trajectory`
`tiago`	`tiago1`	Mobile manipulator	`/tiago1/ joint_trajectory /tiago1/cmd_vel`
`tiago_base`	`tiago_base1`	Mobile base with torso	`/tiago_base1/ joint_trajectory /tiago_base1/cmd_vel`

Derivation from selected YAML entries synchronises renamed namespaces and bridge configuration, while per-instance command topics preserve one-to-one addressing. Thus launch selection, spawned identity and bridge topic all come from the same entry (figure 6.1).

6.5 Joint-command helper

Publishing raw `trajectory_msgs/JointTrajectory` messages is cumbersome and error-prone. The `robot_joint_command.py` operator tool lists supported names, checks the expected position count, maps names to joints, accepts radians or degrees for angular joints, uses metres for linear joints and publishes only to the selected namespace.

Listing 6.1: Examples of the common robot command tool.

```
ros2 run mfja_robot_control_config robot_joint_command.py --list

ros2 run mfja_robot_control_config robot_joint_command.py \
  kuka --unit deg --positions 34.38 -57.30 63.03 0 34.38 0

ros2 run mfja_robot_control_config robot_joint_command.py \
  staubli --unit rad --positions 0.1 0.4 -0.6 0 0.5 0
```

The helper standardises input and creates a short, correctly ordered trajectory command. Trajectory planning, collision-free arm motion, transport-cell synchronisation and controller-specific constraints remain separate.

6.6 Interface portability

Public namespaces and typed state boundaries are independent of the simulator backend. A provider is the runtime component accepting commands or supplying state behind them. Gazebo can therefore be replaced while preserving identity, units, timestamps, freshness and controller-feedback semantics. The reported implementation and results use the Gazebo providers identified in appendix A.

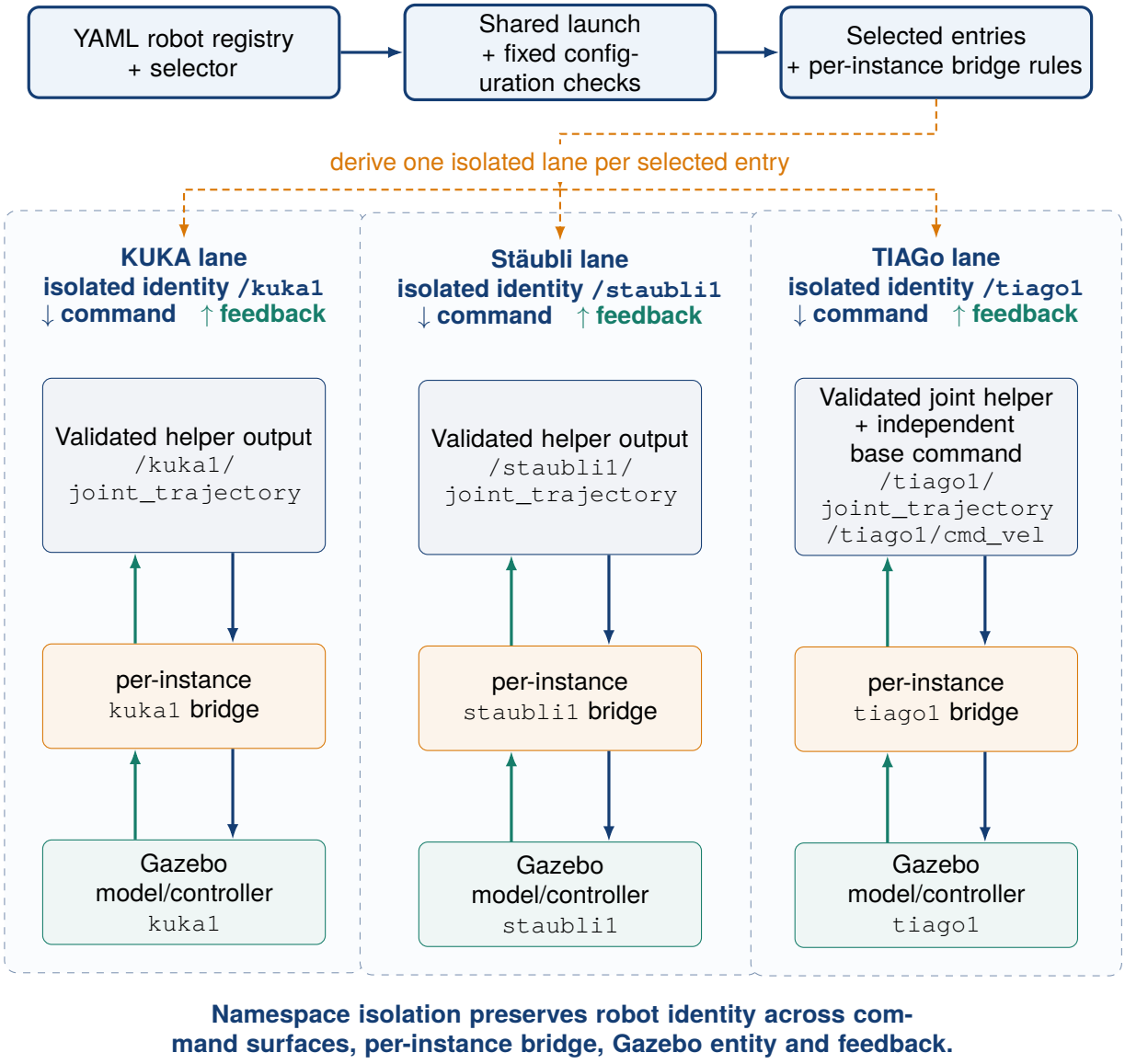


Figure 6.1: Configuration-driven multi-robot launch, command and feedback isolation. Selected YAML entries generate per-instance bridges; the selected operator-helper profile addresses the corresponding name. There is no shared global joint-command topic.

6.7 Results

The selector implements `none`, one exact name, comma-separated sets and `all`. In the retained all-robot cold-start smoke test, the full-floor entry point spawned all six enabled registry entries. Each exposed a distinct joint-command topic and a newly received, post-launch namespaced state sample; both TIAGo variants also exposed independent base-velocity topics. The smoke recorded startup and feedback without motion commands. Named automated suites in chapter 11 cover command-helper and launch-selection rules, including the remaining selector rules. Together, the live check and automated tests show that the separate simulation models share a maintainable access interface.

Chapter 7

Room 315 Kinematic Transport System

7.1 Modelling decision: dynamic or kinematic

At the beginning of the rail implementation, two motion strategies were considered:

Dynamic wheel–rail simulation Model every shuttle as a driven rigid body and let the Gazebo physics engine determine its motion from wheel or guide contacts with the rail, including the changing geometry at switches.

Kinematic rail-following simulation Represent a shuttle by its current graph segment and arc-length coordinate, compute the corresponding pose from the calibrated rail centre-line, and publish that pose to Gazebo.

Dynamic simulation suits traction, contact-load, slip, derailment and wear studies, but here it would add cost without reliable information. Eight shuttles create many coupled wheel/guide–rail contact and friction constraints, especially at switches. A credible model would require validated collision geometry, masses, inertias, profiles, friction, compliance or damping, drives and switch mechanics, potentially with smaller steps and more solver iterations to limit penetration, jitter and contact loss. Without measured parameters, the apparent accuracy would be unsupported.

The required tests instead concern route selection, slot arrival, occupancy, switch/stopper logic, camera perception, planning and supervised execution. Kinematic motion gives repeatable trajectories, sensor crossings and logical progression at lower multi-shuttle cost, generating consistent visual data faster. It also isolates failures: geometry sets the pose, topology the successor, and control/supervision whether motion is allowed.

7.2 Kinematic transport backend

The backend advances arc length on a calibrated graph and publishes the Gazebo pose through the world `set_pose` service, giving the static rail assets predictable routes, speed and sensor crossings for repeatable scenarios focused on planning/supervised execution. Shuttles are kinematic, gravity is disabled and wheel–rail contact does not produce motion. The controller's logical rail state is authoritative; asynchronous pose publication and rendering may lag without changing logical progression.

7.3 From SolidWorks points to a continuous rail graph

Both rail configurations use the same 276 ordered SolidWorks centre-line points in 14 comma-separated-value (CSV) files, then apply side-specific name/topology mappings and calibration. Each row is `index,x,y,z`. Conversion proceeds as follows:

1. split exterior, interior and connector curves at branch boundaries;
2. preserve point order in one CSV per segment, forming a polyline traversed from index 0 to the last row;
3. map each public segment to its CSV and declare its endpoint nodes in network YAML;
4. parameterise arc length and interpolate consecutive points cubically with Hermite tangents estimated from neighbours;
5. apply the calibrated rail-to-Gazebo transform before pose publication.

CSV files define *geometry*; routing tables define connectivity. A *source curve* is an ordered CSV polyline, an *internal geometry segment* its side-specific runtime identifier, and a *public segment* the

canonical ROS/planning label. Twelve anchor nodes identify segment ends at the same physical junction because floating-point equality is fragile. The recorded 0.05 m CAD-to-topology tolerance is metadata, not a runtime routing test.

Each public segment maps to a source curve and has a length. Published shuttle state contains its public segment and arc length; `s_ratio` is the normalised coordinate $\rho = s/L$ from equation (4.1), not another position. The pose evaluator computes

$$\mathbf{p}(s) = \begin{bmatrix} x(s) & y(s) & z(s) \end{bmatrix}^{\mathsf{T}}, \qquad \psi(s) = \operatorname{atan2}\left(\frac{dy}{ds}, \frac{dx}{ds}\right). \tag{7.1}$$

The result is transformed to the Gazebo frame. Cubic Hermite interpolation controls position and tangent at boundaries; linear interpolation creates yaw jumps, whereas an unconstrained global spline may leave the rail. Retained tests load every normalised segment with both backends: `polyline` samples the piecewise-linear CSV, while `cubic_hermite` is the continuous runtime evaluator. Both preserve topology; route/device tests cover successors and slot/sensor mappings. Figure 7.1 compares all CSV points with dense runtime-evaluator samples.

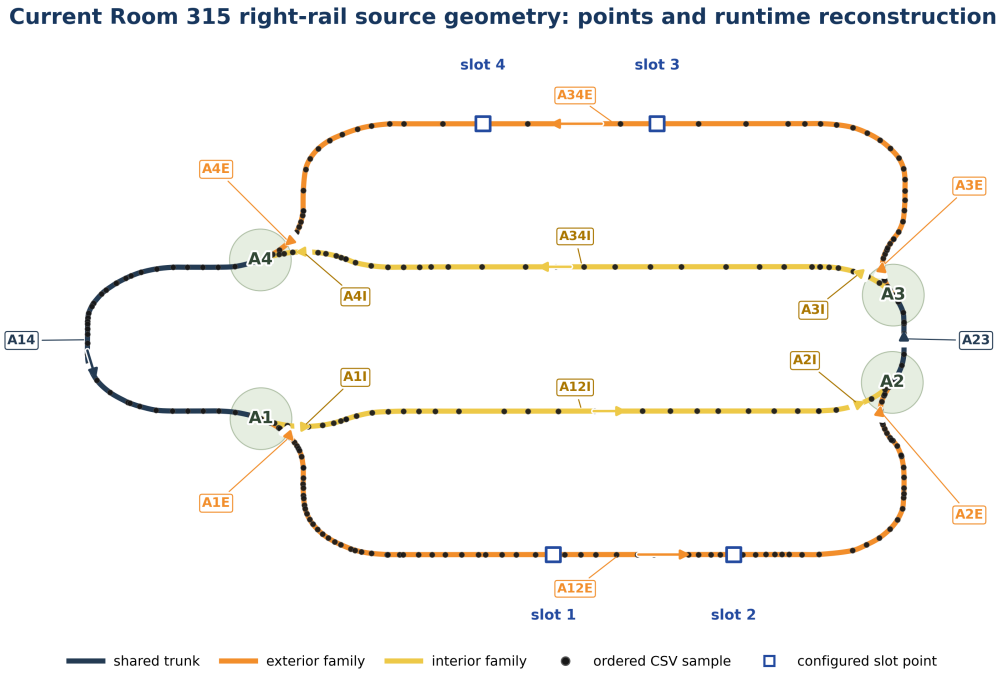


Figure 7.1: Room 315 centre-line from 276 ordered CAD/CSV samples: black dots are source points and coloured curves are dense Hermite outputs. The drawing shows the right-rail source/CAD orientation; the left rail reuses the samples with its own names and calibration. Affine calibration applies side-specific planar scale, rotation and translation plus height/yaw offsets before Gazebo publication. This checks configured reconstruction; geometric accuracy would require independent metrology.

7.4 Topology and switch assignments

Each rail has four exterior/interior switches, hence $2^4 = 16$ assignments. The left/right network YAML files define nodes, segments, connections and this 14-label public vocabulary:

$$\{\text{A12E, A12I, A14, A1E, A1I, A23, A2E, A2I, A34E, A34I, A3E, A3I, A4E, A4I}\}.$$

A1–A4 name physical switch zones; suffixes `E/I` mean exterior/interior. Thus `A3E` is the exterior branch after A3 and `A34E` continues towards A4, while `A23/A14` are shared single-track segments. These are interface labels, not extra rails.

The controller and PDDL generator read this same topology, including blocker clearance beyond the outer loop. Table 7.1 uses only public ROS names. On the left rail, internal geometry identifiers are converted to public segment labels; switch identifiers are already public. Both rails therefore share the public successor contract.

Table 7.1: Public successor map shared by both Room 315 rails.

Current segment	Route switch	Switch state	Next segment
A23	A3	E	A3E
A23	A3	I	A3I
A3E	—	Any	A34E
A3I	—	Any	A34I
A34E	A4	E	A4E
A34E	A4	I	FALLING
A34I	A4	E	FALLING
A34I	A4	I	A4I
A4E	—	Any	A14
A4I	—	Any	A14
A14	A1	E	A1E
A14	A1	I	A1I
A1E	—	Any	A12E
A1I	—	Any	A12I
A12E	A2	E	A2E
A12E	A2	I	FALLING
A12I	A2	E	FALLING
A12I	A2	I	A2I
A2E	—	Any	A23
A2I	—	Any	A23

The table, not point distance, selects successors; “Any” is a fixed transition. An incompatible converging switch has no successor and enters `FALLING`: a latched no-valid-successor fault that stops at the segment end, not a physical fall. In forward traversal A2 enters `A23`, then A3 selects its exterior/interior successor (figure 7.2).

7.5 Devices and public naming

YAML devices have public names grouped by switches, branches and indexing zones. A DZI name is `DZI` + slot + rail suffix `R/L`; for example, `DZI1R` detects right-rail slot 1. DA switch-approach names use switch number, optional branch `E/I`, then rail suffix; no branch suffix means the switch’s single-track side. Figure 7.3 shows the right rail.

7.5.1 Switches

Switches A1–A4 select exterior/interior branches through separate command and feedback topics. Updates do not pause Gazebo globally. The protected region extends 0.35 m along the rail; chapter 8 defines the feedback-based veto.

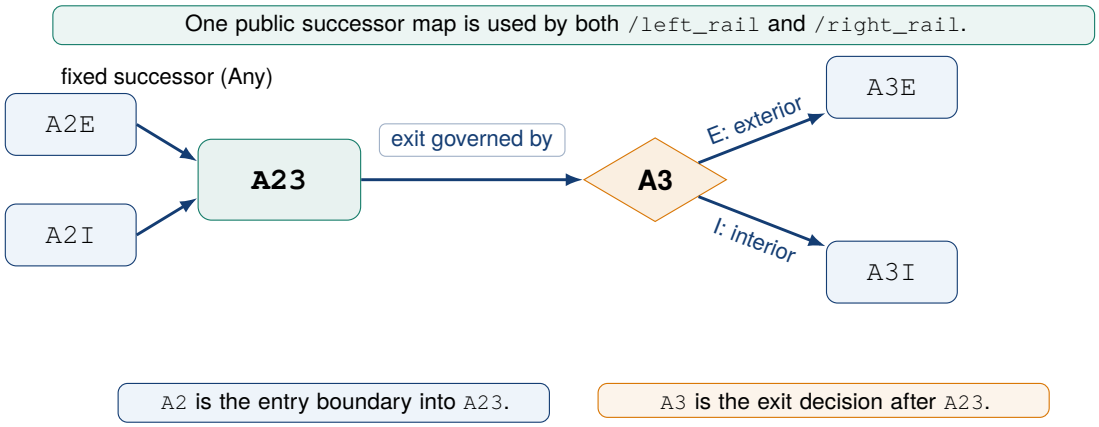


Figure 7.2: A2 enters public segment A23; A3 then selects its exterior/interior successor. Both rails use this public rule.

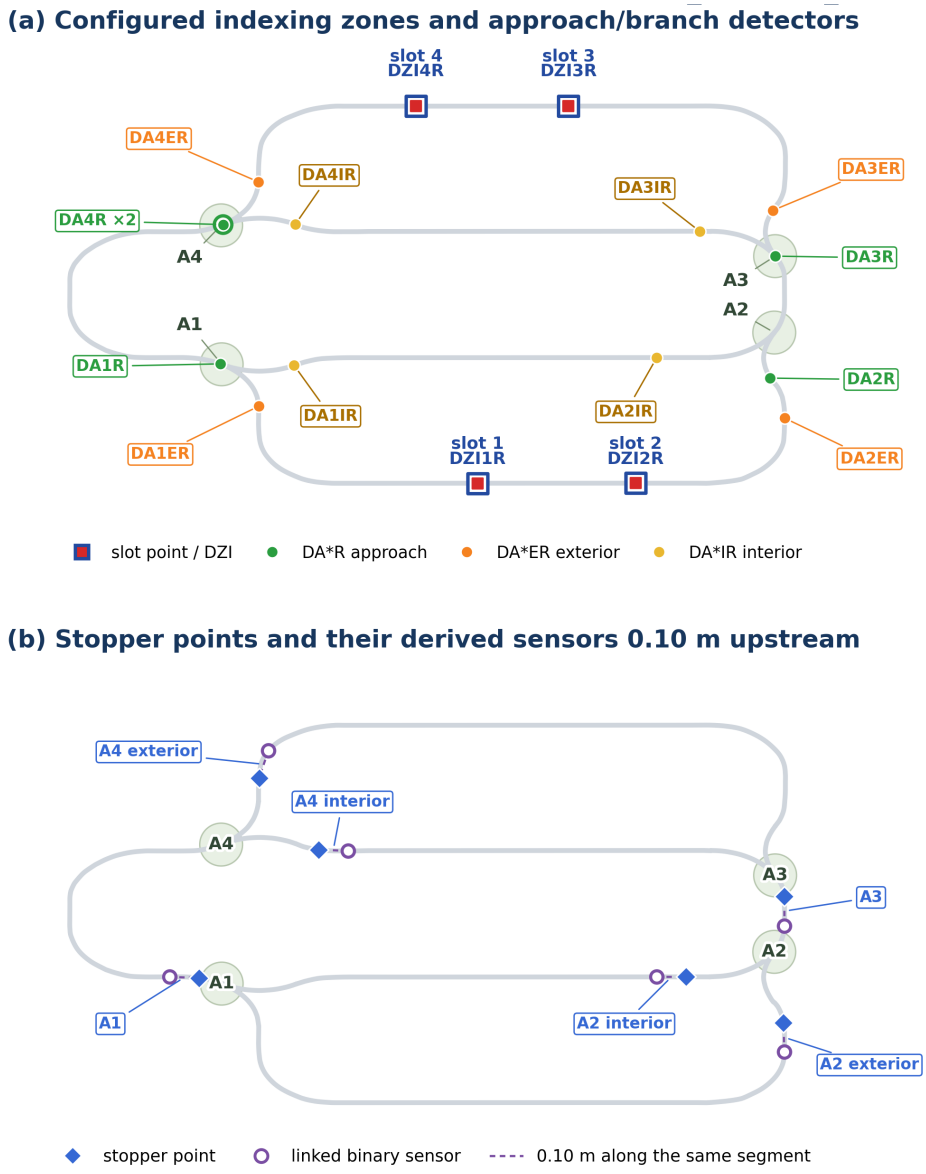


Figure 7.3: Configured right-rail devices on the controller's Hermite geometry: panel (a) shows slots and approach/branch detectors; panel (b) separates stoppers from linked binary sensors 0.10 m upstream.

7.5.2 Stoppers

Stoppers hold/release a shuttle before protected areas or targets. Controller state `0` means open/released and `1` closed/stop. The workflow closes the target stopper, opens upstream ones, starts motion and awaits the identity-bearing target sensor. Because a stopped shuttle may retain a nonzero speed setting, motion is determined from controller mode.

7.5.3 Binary sensors

Sensors stream binary occupancy, not distance: `active=0/1`, with a separate `shuttle_name` when active. DZI detectors identify slot arrivals; approach/branch detectors locate a shuttle near a switch or inner/outer route. In figure 7.3, $\times 2$ means one logical detector has two configured points. Continuous pose remains controller-only for safety/evaluation, not a learned-model input.

7.6 Multi-shuttle identity and lifecycle

Each rail supports four stable identities: L1–L4 or R1–R4, mapped to Gazebo names such as `room315_right_shuttle_4`, never inferred from array order. The typed add/spawn service accepts name, start slot, speed and initial enabled state. Per-entity commands are ON (enable/resume), OFF (disable), RESET (return to configured start slot) and REMOVE (delete from Gazebo); `ALL` applies ON/OFF/RESET rail-wide. Reports expose MOVING, WAITING, DISABLED and FALLING modes, payload state and debug identity. Emergency stop disables all motion; resumption requires an explicit operational decision.

The identity registry reconciles fleet limits, configuration, world entities and default spawns. Added entities enter supervised execution only when they match its L1–L4/R1–R4 language-interface identities, keeping ad hoc commissioning outside the planner domain and aligning visual/planner objects. Figure 7.4 shows the full fleet in both visual-pipeline cameras.

7.7 Motion, slots and occupancy

At every update step, an enabled shuttle advances by

$$s_{k+1} = s_k + v\,\Delta t, \tag{7.2}$$

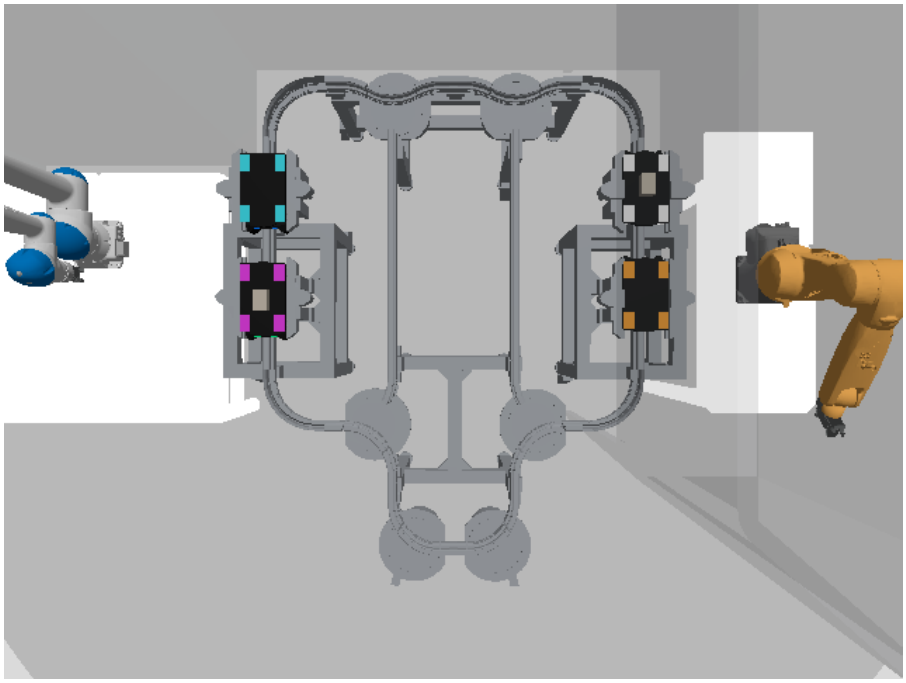
where s_k is arc length at update k, v configured forward speed and Δt the interval. Route, target, stopper and headway constrain the update; crossing an end preserves residual distance on the topology-selected successor. Controller and Gazebo `set_pose` rate limits default to 30 Hz. Device logic follows controller timing, while asynchronous pose requests may reduce visual update rate.

Slots are calibrated exterior-segment `s_ratio` values. Arrival combines target, stopper, slot detector and OFF command effect: OFF reports DISABLED, while `reached_target_slot` records low-level setpoint completion, not task success. The planner tracks symbolic occupancy; the executive still verifies target identity.

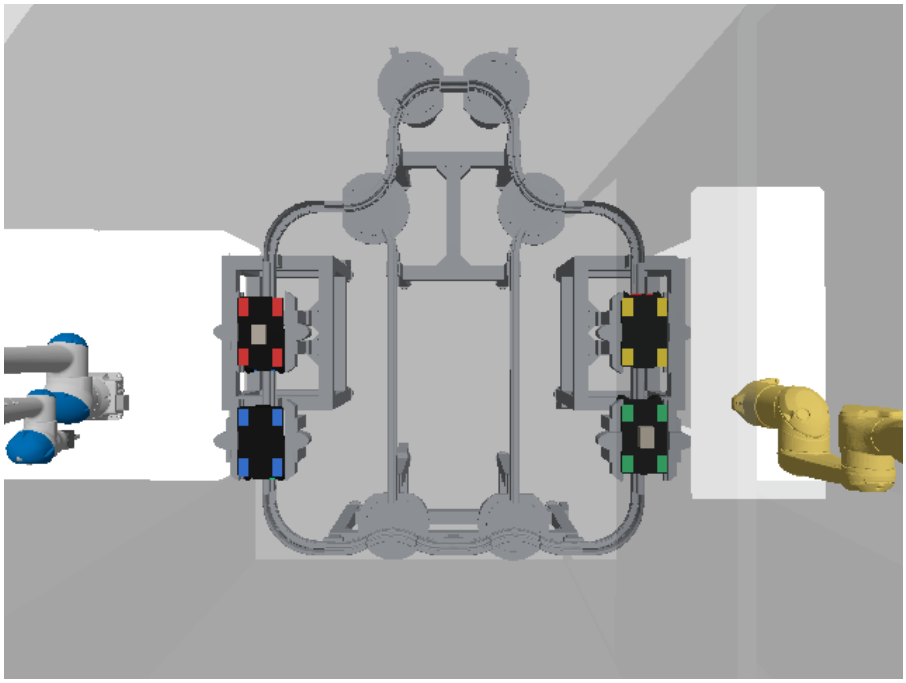
7.8 Operator-facing observation and maintenance

The typed low-level API in `runbook.html` supports commissioning and device-isolation tests; operational tasks still pass through translation and supervision.

The installed device-position tool projects Gazebo XYZ onto the nearest segment of the YAML device geometry and reports arc length, `s_ratio`, projected point and residual, defaulting to dry run. Guarded writes update slots, approach/branch detectors or stopper points; over-threshold residuals require explicit override. A logical multi-point stopper may protect corresponding points on two branches, addressed by segment or list index, while its linked detector remains derived.



(a) Left cell: L1 cyan, L2 magenta, L3 orange and L4 white. The centre payload blocks are visible on loaded L2 and L4.



(b) Right cell: R1 red, R2 blue, R3 green and R4 yellow. The centre payload blocks are visible on loaded R1 and R3.

Figure 7.4: Synchronous overhead Gazebo views of all eight shuttles. With device markers hidden, identity colours, presence and payload appearance remain visible.

Changes are offline until relaunch and marker/binary-feedback checking. Commands remain separate from closed-loop tasks and are documented in `runbook.html` and `docs/QUICK_START_AND_FEATURE_GUIDE.md`.

7.9 Headway and collision constraints

The low-level controller enforces configurable geometric separation, default minimum centre distance 0.33 m. Higher-level *block headway* is the required number of clear route blocks ahead; chapter 8 defines reservation ownership and authorisation.

A runtime *blocker* is another shuttle occupying the requested route, destination slot or holding capacity. Other rail objects are outside the planning/execution model.

7.10 Interior staging and dense blocker cases

Public interior branches `A12I/A34I` are capacity-limited holding areas. From calibrated length, the planner computes rather than fixes centres, keeps a 0.35 m end margin and configured separation, and prefers the farthest reachable free centre. Each branch currently admits at most two safe shuttles. Successful supervised staging persists a *runtime clearance certificate* binding identity, interior target, entry sensor, bounded motion, controller stop and post-stop observation. It supports safe route normalisation, not visual localisation, and a later `ON` invalidates it. These centres constrain relocation; chapter 10 covers selection, re-observation and recovery.

7.11 Validation of the transport model

Static/behavioural checks cover geometry, topology, devices, identity, separation and interior capacity (chapter 11); typed commands and supervision enforce these constraints (chapter 8).

Chapter 8

Typed Interfaces, Contracts and Safety Supervision

8.1 A typed command and state interface

The typed rail interface separates command/state topics; every message defines its role, required fields and device, keeping requests distinct from reports.

ROS 2 `.msg`, `.srv` and `.action` files [15] define named state, switch/stopper/shuttle command–state, sensor and visual messages, plus `AddShuttle`, independently of Gazebo implementation.

8.2 Command/state separation

Three roles are distinct: a *command* is a request; *state* is the controller report; and a *result* records status, supporting readings and, where applicable, a reason.

A *command effect* is expected post-publication controller/sensor state, which the executive observes rather than assumes. It differs from a PDDL symbolic effect, the planner's predicted change. Exact arrival needs post-command DZI and controller reports; switch/stopper checks apply their freshness rules. A report is *fresh* only while its timestamp age is within the configured limit.

8.3 Sparse partial-update semantics

Live switch/stopper commands are sparse named updates. A *checked command envelope* is the validated JSON-compatible request before typed-message conversion. Thus `switches: {A3: INTERIOR}` changes only A3; omission means preserve state, not apply a default.

After safety checks, the supervisor publishes `SwitchCommand` or `StopperCommand`; their `Named-State[]` arrays apply only present entries. Motion uses `ShuttleCommand`. The translator's `event_action` is non-actuating primitive/target metadata; training-only `target_mask` marks available label values for loss. Neither is a live command.

Figure 8.1 summarises feedback confirmation and named-only change.

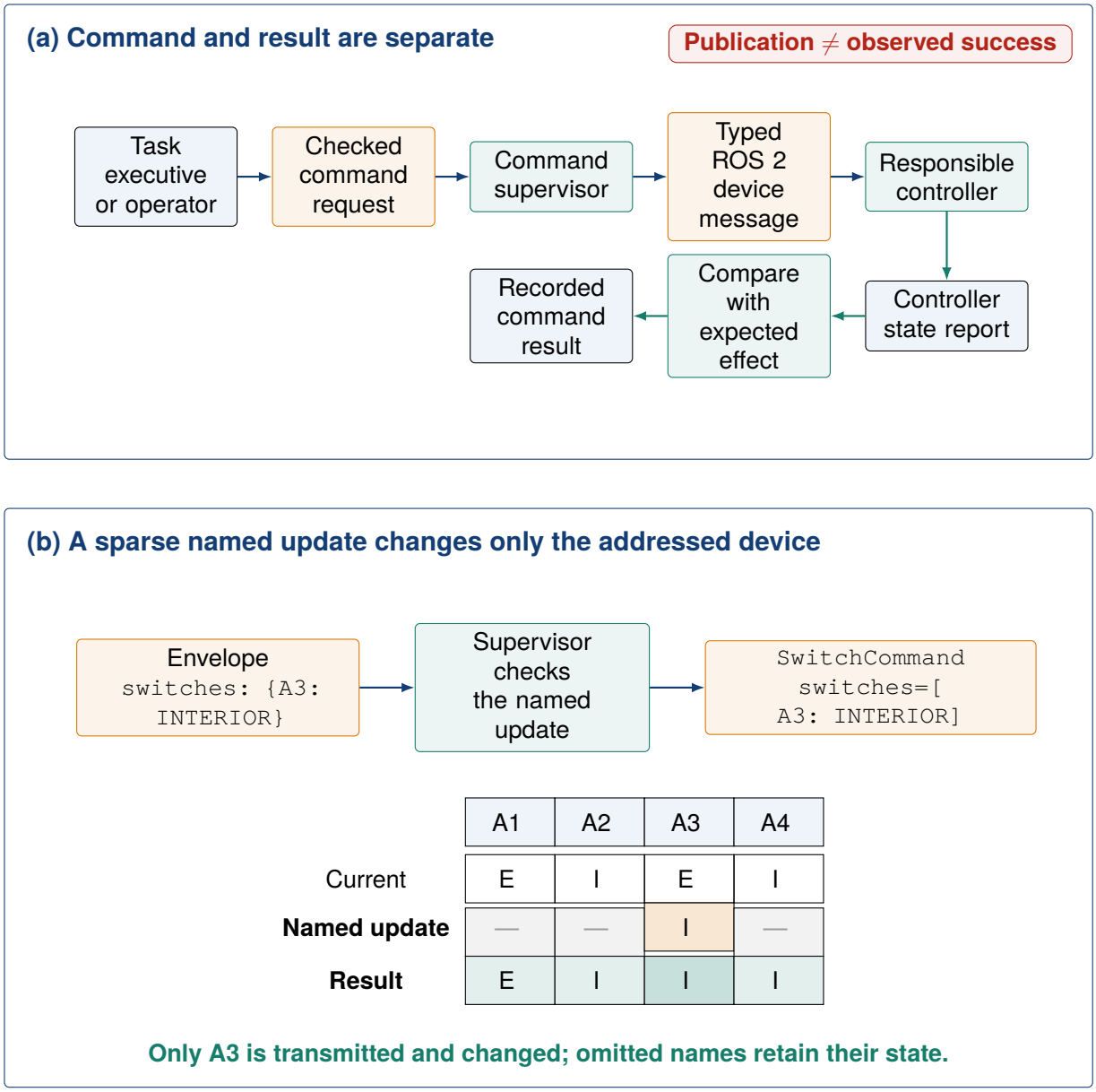


Figure 8.1: The supervisor checks sparse commands before typed publication; feedback establishes results and omitted devices retain state.

8.4 Versioned contracts

Operational/task data use versioned Python contracts: named schemas with validation, provenance and allowed producers. Records carry a schema version; incompatible versions are rejected. Planning/execution combine typed records with checked JSON commands and, where required, identifiers, timestamps and producer/source data. The supervisor converts authorised requests to typed ROS 2 messages. Table 8.1 lists the principal contracts.

Table 8.1: Principal closed-loop contracts and their producers.

Contract	Meaning	Allowed producer
`ObservedFact`	Value, confidence and known/unknown/stale/conflicting status	Visual or rule-based provider, identified by source
`ObservedState`	Visual inputs and fused facts with freshness/status metadata	State-fusion layer
`SemanticParseEnvelope`	Strict user-text semantic proposal	Local language model; semantic fields only, excluding PDDL/plan/command
`TaskGoalDraft`	Incomplete proposal after field precedence/fusion	Explicit parser and field-fusion resolver
`TaskGoal`	Immutable validated constraints (unchanged after publication), optionally grounded to one current instance	Domain validator; gateway may ground from current visual facts
`PddlPlanStep`	One returned-plan symbolic action	PlanSys2 boundary; only the executive may add a required route-normalisation prerequisite
`TranslatedPlanStep`	Parsed action, sparse command and event-action record	Plan translator
Checked command dictionary	Actuation intent with named updates only	Executive/macro dispatcher; supervisor-authorised before publication
`SwitchCommand/` `StopperCommand/` `ShuttleCommand`	Typed execution messages; devices carry sparse `NamedState[]`	Rule-based supervisor after current-state validation
`PostconditionCheck` and closed-loop records	Effect status, reason and step/task outcome	Executive using supervisor/controller/observation reports

Producer means the component allowed to construct a contract; *source* is stored fact provenance and controls use. Thus simulator pose may label or veto but never enters visual-model input. Language proposals require fusion, validation and operator confirmation. Field fusion combines structured input, explicit text, confirmed context and semantic proposal by fixed precedence, recording disagreements. The grounding gateway resolves *any*/*nearest* against current state; route normalisation restores the exterior route after special clearance; and a macro dispatcher deterministically expands an approved macro into checked commands. Producer boundaries appear in table 8.1; the full sequence is in figure 10.1.

8.5 Observed-state semantics

Critical facts preserve four statuses rather than coercing them to false: *known*; *unknown* (unobserved); *stale* (freshness elapsed); and *conflicting* (authorised sources disagree). Figure 8.2 shows their planner/actuation eligibility.

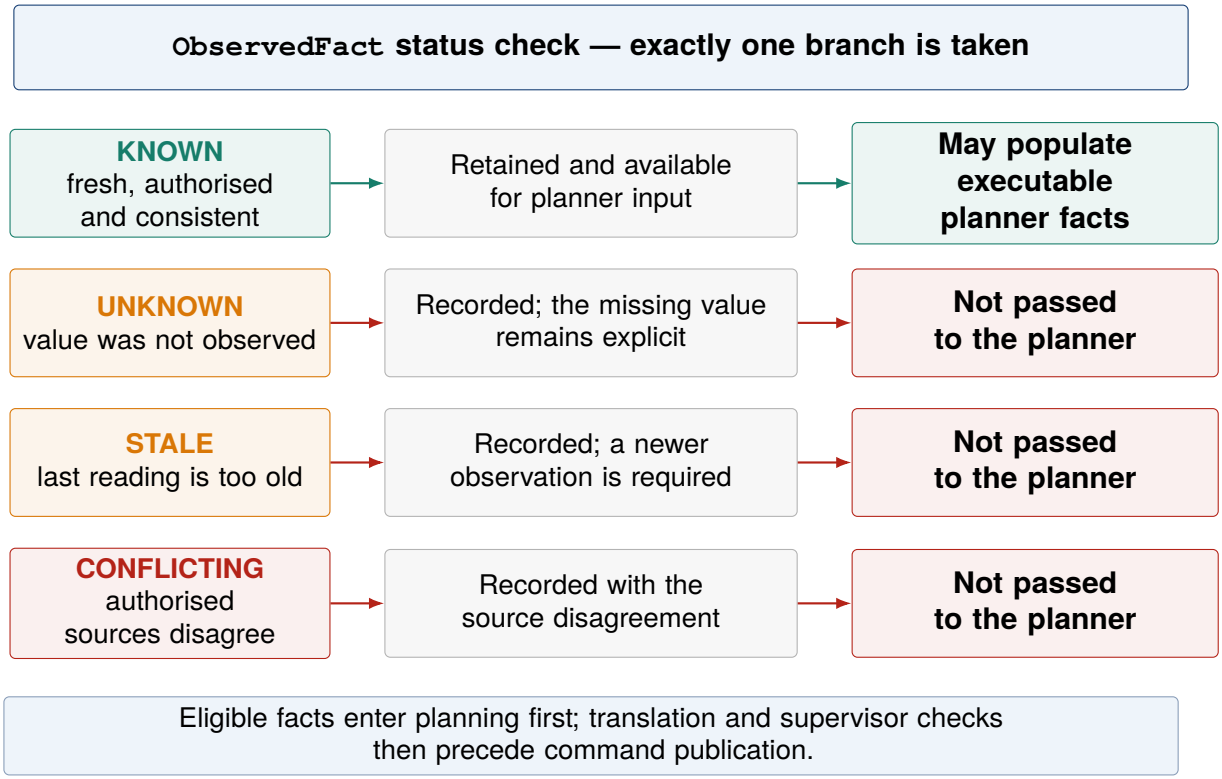


Figure 8.2: Status handling before an observed fact enters planner state.

`ObservedState` separates:

`visual_model_inputs` visual-model-origin facts before source fusion; despite the legacy field name, these are learned outputs rather than the raw images supplied to inference;

`fused_planner_state` fused facts with their status and freshness metadata after source reconciliation.

The live task gateway combines a confirmed task with current state, retaining the latest accepted visual, supervisor and device reports. A provider supplies one current executive snapshot. Live operation separates oracle and visual providers; test-only providers remain outside it.

8.6 Supervisor responsibilities

The rule-based `room_315_vla_supervisor.py`—not a learned VLA policy—gives final software authorisation. It validates sparse JSON commands against rail state, then publishes typed commands consumed by the kinematic controller.

The supervisor gates include:

- latched emergency-stop state and the immediate global `stop_all` operation; command structure, identity and rail-side consistency;
- fresh/non-conflicting state, occupied targets, reservation-block ownership, geometric separation and block headway;
- switch change within the 0.35 m protected distance; stopper and route-clearance state;
- post-command occupancy by another shuttle, sensor dropout, timeout, unknown localisation, impossible pose or `FALLING` mode;
- a configured recovery-retry limit (default two) and lexicographic reservation-conflict tie-breaking.

Diagnostics retain metrics, recent decisions and each authorisation/rejection reason; rejection leaves the device unchanged. Exceptions or shutdown during an active goal trigger gateway `stop_all`.

8.7 Reservations and deadlock

A *reservation block* is a mutually exclusive rail-side/public-segment route resource, temporarily assigned to one shuttle/task; target-slot conflicts are separate. Reservations prevent two execution primitives (translated operations) entering it; geometric separation independently prevents overlap.

Occupancy by a non-owner is a crossed-ownership deadlock conflict. The lexicographically first identity gains priority; others stop, observe and plan again. This detector covers crossed conflicts, not temporal non-progress or larger wait-for cycles (each participant awaiting the next one's resource). Recovery still needs a planned reachable destination, capacity and effect check.

8.8 Source of switch feedback

The planner uses visual location, while switch safety independently checks continuous controller coordinates. It maps the public controller segment to the side-specific internal metric graph and measures distance along rail, not Euclidean space. Only current, finite, graph-mappable state authorises a switch; fail-closed handling inhibits all else.

8.9 Checking command effects

The runtime defines the expected effects of every translated execution command. Examples include:

- `SET_SWITCHES/SET_STOPPERS`: requested values match controller status;
- commanded slot motion: two consecutive identity-bearing DZI readings, controller `reached_target_slot`, explicit `DISABLED` state and required confirmation frames agree;
- `STOP_NOW`: controller reports a fresh explicit `DISABLED` state;
- observation-only inspection: a newer visual observation is available after the request.

Results retain command, plan-step and observed-state identifiers; motion effects also record command-linked observations and require newer reports for arrival and retries. Switch/stopper matching permits *cached state*: the last accepted value may predate the command, unlike post-command motion checks.

8.10 Manual and emergency paths

Manual device-debug commands use normal structural/supervisor checks. The exception is emergency: it immediately invokes all-shuttle `stop_all` without planner state and latches rejection of motion until explicit clearing. Stoppers can thus be tested without a language model but remain safety-gated.

Chapter 9

Developing the AI Inputs: English Understanding and Visual-State Learning

9.1 Phase 2 objective and implementation sequence

Phase 2 added two AI inputs to Room 315: English requests become confirmed `TaskGoal` records, while paired cameras produce checked `VisualStateObservation` records for planning shuttle motion. The latter is a versioned paired-frame record carrying fixed-identity estimates, image timestamps, schema/checkpoint identity, validation status and provenance (producer and source). These typed records, not free text or raw images, form the planning boundary.

9.2 Reused components and project-specific development

The language model was integrated pretrained and quantised, without Room 315 fine-tuning; the visual network reused pretrained features and was then trained on the project's paired-camera corpus.

Table 9.1: Reused foundations and project-specific Phase 2 work.

Component	Reused foundation	Developed in this project
English understanding	Pretrained Qwen2.5-1.5B-Instruct weights and the local `llama.cpp` inference backend	Saved local configuration, constrained prompt, strict JSON schema, explicit-fact extraction, field fusion, clarification, validation and confirmation dialogue
Visual state	ImageNet-pretrained TorchVision ResNet-18	Gazebo corpus, rail-isolated paired-view architecture, 168 learned values, masked multi-head loss, calibrated segment confidence, grouped evaluation and hash-bound runtime integration
Planning	PlanSys2 and its POPF solver	Room 315 PDDL model, fresh-problem generation and closed-loop integration (chapter 10)

9.3 Building the English task interface

9.3.1 Local model selection and configuration

The selected semantic model is `Qwen2.5-1.5B-Instruct-GGUF` [18, 24], specifically `qwen2.5-1.5b-instruct-q4_k_m.gguf`: GGUF is the local file format and Q4_K_M its four-bit quantisation. It runs offline on the CPU through `llama.cpp` [4], with a 4,096-token context, temperature 0, 192 generated tokens at most and seed 315; its hash is in appendix A.

Without fine-tuning, a versioned prompt constrains the model to one `SemanticParseEnvelope` JSON object over permitted task fields, identities, rails, stations and slots; it excludes PDDL, plan steps and device or shuttle commands.

9.3.2 From model output to a confirmed task

A rule-based extractor records explicit facts (for example, shuttle colour, rail or slot), Qwen proposes intent fields, and fixed-precedence fusion prevents the proposal from overwriting explicit or

confirmed context. The resulting non-executable `TaskGoalDraft` is clarified, domain-validated and confirmed before an immutable `TaskGoal` is issued.

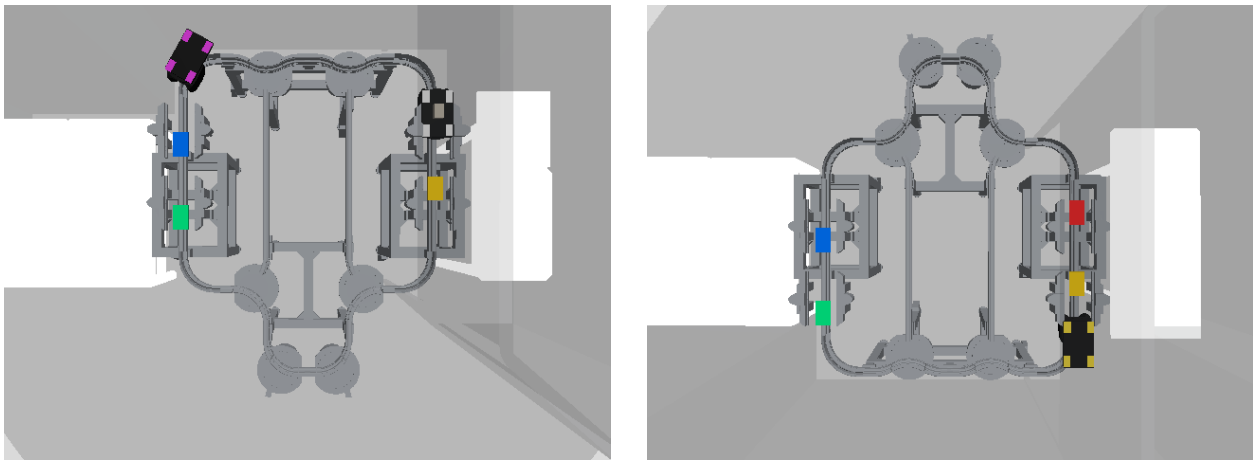
For the indirect request "Could you get a loaded shuttle over to the right slot 4?", the saved record shows the model supplying `goal_type=transport` and deterministic fusion/validation constraining side, payload and destination. The ten-case evaluation is reported in section 11.3.1.

9.4 Generating the Room 315 visual corpus

9.4.1 Paired-camera corpus and partitioning

The generator enumerated all $2^8 - 1 = 255$ non-empty subsets of identities L1–L4/R1–R4 and produced eight geometrically varied scenes per subset by changing valid rail positions, payload states and switches. After each Gazebo scene settled, it captured one synchronised image per overhead camera: 2,040 paired observations, or 4,080 RGB files. Figure 9.1 shows one input.

Count units are: *presence configuration*, the subset declared present; *scene/scenario/record*, one labelled paired-camera row; *image*, one camera file (normally two per row); and *visible-shuttle target*, one present identity with complete supervision, including a valid projected box (the per-shuttle metric denominator).



(a) Left-camera RGB input

(b) Right-camera RGB input

Figure 9.1: The two synchronised RGB images form the complete learned input for one scenario. Simulator state is used to construct labels in a separate file and is never appended to the image tensor.

Gazebo identity, rail coordinate, payload state and projected box supply labels stored separately from image references. Only paired RGB images are model features; depth, controller state, point clouds and other oracle metadata remain label-generation data.

Historical grouped protocol. All eight variants of each presence configuration stayed together. The predecessor split assigned 191 configurations/1,528 scenarios to training and 32/256 each to Grouped Validation and a held-out Test. Grouped Validation images/labels were evaluated every epoch; their weighted task-loss sum selected epoch 14 and drove patience, though all 15 epochs ran. The Test stayed outside training/selection, then was unlocked and evaluated once on 30 July 2026 after selection, supplying the predecessor acceptance record rather than the current claim. It is therefore a consumed historical Test, not a still-unseen "locked partition", and current loaders denylist its recorded paths and hashes. Neither 256-scenario partition directly enters V4. Grouped Validation has only indirect lineage: its epoch-14 V3 joint-view checkpoint initialised the V4 rail-isolated backbone, but its heads were not copied. Both partitions remain only as historical reproducibility and audit records.

9.4.2 Current training, validation and Canary roles

The current pool combines 1,528 legacy grouped-training scenes with 4,000 training-only hard cases. Each deterministic epoch samples 4,000 weighted records with replacement from each source; weights favour rare side–segment cells (rail side paired with public segment), boundaries, switch zones and multi-blocker scenes. A *hard case* deliberately stresses such factors; it was generated, not mined from Final Test errors.

The separate hard-case Validation (512 scenes; 2,254 present targets) selected V4, drove patience and supplied temperature/segment-threshold fitting. Only after selection, the Development Canary (256 scenes; 1,085 targets) checked the frozen candidate for degradation; it affected neither selection nor calibration and is not Test evidence. Accessing the current Final Test remained prohibited throughout development; it supplies the principal evidence below.

9.5 Designing and training the visual-state estimator

The *visual model* is the learned network; the *visual-inference runtime* adds preprocessing and observation validation; the *closed-loop runtime* further adds the task gateway, planner, supervisor and controllers.

Evolution of the visual pipeline. The early versions primarily evolved the data contract: V1 established paired-camera inputs while keeping simulator-derived labels outside the model input, and V2 added continuous along-rail localisation. V3 fixed the representation to the eight known shuttle identities, resized both cameras to 224×224, concatenated global features from a shared ImageNet-pretrained ResNet-18 and predicted 200 values through one head. This design distorted the native $4{:}3$ images, allowed cross-rail influence and learned side, s_m and segment length even though identity and topology determine them. V4 addresses these limitations with 320×240 inputs, weights shared only through `layer3`, independent `layer4`/spatial-query branches for each rail and 168 learned values, while deriving topology-known quantities deterministically. V4 is the active visual model used by the evaluated runtime. Results are in section 11.2.

9.5.1 Rail-isolated 168-value learned contract

Each $4{:}3$ image is resized to 320×240, ImageNet-normalised and processed separately by shared ResNet-18 weights through `layer3`. Each rail then has its own `layer4` and four identity-query heads; no camera feature enters the other rail's late branch.

The fixed identity order remains L1–L4 followed by R1–R4. Each identity has 21 learned values:

$$21 = \underbrace{14}_{\text{segment logits}} + \underbrace{2}_{\text{loaded-state logits}} + \underbrace{4}_{\text{bounding box } (x,y,w,h)} + \underbrace{1}_{s_{ratio}}, \qquad 8 \times 21 = 168.$$

A logit is an unnormalised pre-softmax score; a box is normalised top-left x, y, width and height. The learned coordinate is that of equation (4.1): $s_{ratio} \equiv \rho = s_m / L_{segment} \in [0, 1]$, where s_m is the within-segment arc length in metres and $L_{segment}$ is that segment's length.

Identity determines side (L1–L4 left; R1–R4 right); after decoding the segment, the runtime reads its fingerprinted topology length and computes $s_m = s_{ratio} L_{segment}$. A derived 200-value compatibility vector is diagnostic only and excluded from control.

Presence is an independent input, not a prediction. Training masks any identity that is absent or lacks a required target. At runtime, every present identity must pass artefact, camera-order, freshness, vocabulary, numeric and confidence checks; any failure rejects the whole observation rather than publishing a partial state.

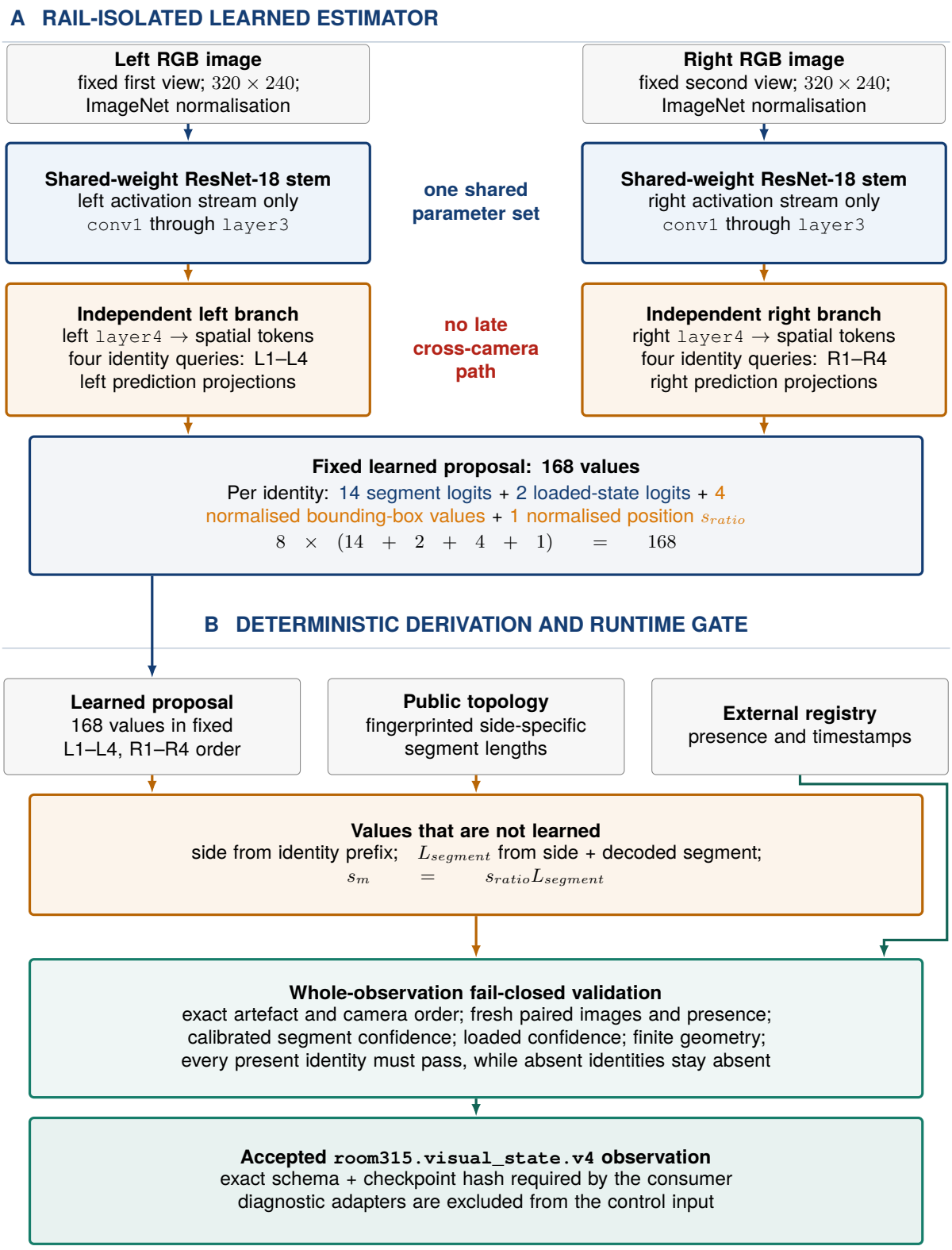


Figure 9.2: The model shares low-level ResNet-18 weights through `layer3`, then keeps `layer4` and the four-query prediction head independent for each rail. The model produces 168 learned values; side, segment length and s_m are derived deterministically before whole-observation validation.

9.5.2 Training and checkpoint selection

Verified predecessor weights initialised the shared stem through `layer3` and each independent `layer4`; no joint head transferred, and rail-branch parameters, BatchNorm buffers and query heads were optimised independently.

Training unfroze heads in epoch 1, independent `layer4` branches in epoch 2 and shared `layer3` at a lower rate in epoch 4. The objective weighted segment cross-entropy 4, s_{ratio} Smooth-L1 2, loaded-state cross-entropy 1, box L1/generalised-IoU 1 and graph-distance topology loss 0.25. Brightness, contrast, saturation, gamma, blur and noise augmentations were allowed; side-breaking flips and camera swaps were forbidden. Exact ranges and the selected configuration follow.

Table 9.2: Configuration of the selected visual training run.

Item	Configured value
Input	Fixed left–right RGB order; aspect-preserving bilinear resize to 320×240; ImageNet normalisation
Architecture	Shared ResNet-18 through `layer3`; independent `layer4` and four-query spatial head per rail; no cross-camera feature path
Optimiser	AdamW; head, `layer4` and shared-`layer3` learning rates 3×10^{-4}, 10^{-4} and 2×10^{-5}; weight decay 10^{-4}
Batch and schedule	Batch size 24; 12 epochs; mixed precision; staged unfreezing; early-stopping patience 4
Loss	Segment 4; s_{ratio} 2; loaded state 1; bounding box 1; topology 0.25; presence-aware masks
Selection	Lexicographic validation planning score, worst side–segment recall, then correct-segment s_m 95th percentile (p95)
Reproducibility	CUDA training, seed 31520260811; Final Test unavailable and forbidden during training, selection, calibration and threshold selection; Canary excluded from training and selection

All 12 epochs completed and epoch 11 was frozen (identity in appendix A). Here, an epoch is 8,000 sampled records, not one unique pass through the 5,528-scene pool; a checkpoint is the saved weight state after an epoch. Comparison was lexicographic: first the weighted planning score $0.30F_{1,\text{macro}} + 0.15A_{\text{min side}} + 0.20R_{\text{worst cell}} + 0.20A_{\text{joint},0.05} + 0.075A_{\text{switch}} + 0.075A_{\text{boundary}}$ (segment macro F1, lower-side top-1, worst-cell recall, joint segment/coordinate accuracy at $|\Delta s_{ratio}| \leq 0.05$, and switch-/boundary-zone top-1); then worst side–segment recall; then the lowest correct-segment s_m p95. Four unimproved epochs would stop training; this patience 4 condition was not reached.

Hard-case Validation fitted the segment temperature 0.6007370031399095 (dividing logits before softmax) and then the deployed score floor 0.9139089813389109 from its retention–accuracy curve. Loaded state uses an uncalibrated maximum-softmax floor of 0.5, not a probability guarantee. The runtime binds checkpoint hash, architecture, preprocessing, camera order and segment-length fingerprint before becoming ready; section 11.2 distinguishes these online floors from labelled offline tolerances.

9.5.3 Preregistered Final Test

Before evaluation, the checkpoint, scenario plan, evaluator, coverage, thresholds and single-run policy were frozen. The frozen Final Test comprised 1,040 scenes, 2,080 images and 4,680 visible targets, and was executed once after model selection. *Preregistered* here means the repository controls were frozen before any Test prediction, not registered externally. One normalised control schema and hashes then established byte-identical captured data before the attempt lock.

Independence/disjointness covers procedure, scene IDs, canonical generation-parameter fingerprints, image bytes and complete label rows, not presence configurations; abstract task factors intentionally recur.

After these checks, a single-use attempt lock and completion ledger were created and the checkpoint ran once, without training, selection, calibration refit or threshold selection. This evaluation lock is unrelated to rail-route reservation. Results, development-partition metrics and synthetic-camera limits are consolidated in section 11.2.

9.6 Integration outputs and runtime qualification

Integration exposes only a confirmed `TaskGoal` and current `VisualStateObservation`, carrying the scores, schema and checkpoint hash required by the gateway. Chapter 10 follows their conversion to planning facts; section 11.2 defines qualification and authority lifecycle. In particular, node-local `dry_run_state_fusion` blocks only the redundant ProblemExpert mirror: the separately activated gateway builds PDDL problems and owns Gazebo-only actuation.

Chapter 10

From an English Instruction to Shuttle Motion

10.1 Runtime architecture

Figure 10.1 follows English and camera inputs through validated proposals, PlanSys2/POPF planning, supervised typed commands and verified Gazebo effects; an incomplete goal re-enters observation and planning. Interface rules are in chapter 8.

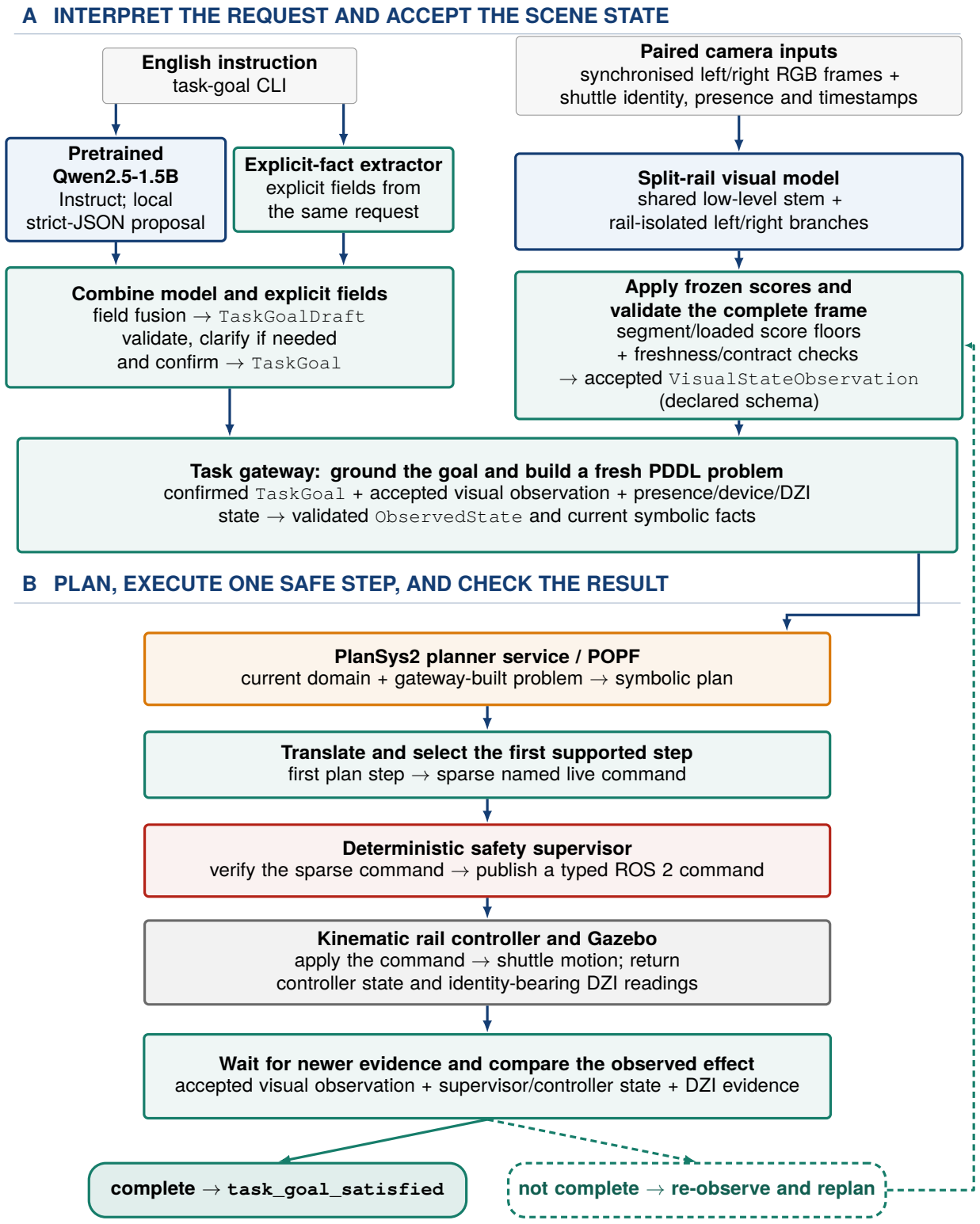


Figure 10.1: End-to-end sequence from task and paired-camera inputs to a verified Gazebo result; the dashed green path denotes observation and replanning while the grounded goal remains incomplete.

10.2 Step 1—interpret and confirm the English task

The CLI applies chapter 9, clarifies missing fields and requires confirmation before publishing `TaskGoal`. It resolves explicit identities and colour aliases; scene-dependent policies remain ungrounded. *Grounding* selects one shuttle and, for station requests, an admissible slot; the planner, not the language model, chooses actions.

10.3 Step 2—read the current scene and ground the goal

The visual runtime verifies its manifest—the hash-bound artefact, schema, configuration and Gazebo-scope record—plus checkpoint, preprocessing and camera order. Any missing, stale, conflicting or low-confidence requirement rejects the pair; an accepted `VisualStateObservation` carries schema and provenance. The inference node does not mutate ProblemExpert: the gateway fuses the observation with presence, device and identity-bearing sensor reports into `ObservedState` for planning.

Symbolic shuttle locations are derived directly from the accepted visual observation rather than from controller coordinates.

The gateway validates and grounds *any*, *nearest* and station-slot policies against current state/occupancy. Payload-based selection requires, by default, five distinct consecutive accepted observations within freshness limits. The grounded identity retains the goal ID and stays fixed for execution.

10.4 Step 3—build a fresh PDDL problem and plan

10.4.1 From current state to decision predicates

Each cycle builds a fresh PDDL problem from the grounded goal and latest `ObservedState`: present shuttles, locations, occupancy, payload and device facts. PDDL receives no raw image/text; unknown, stale or conflicting state blocks generation rather than implying free space.

10.4.2 How the current facts constrain the next action

Transport actions require `validated_state`, matching `active_goal_side`, and action-specific location, occupancy and route predicates. POPF minimises dimensionless symbolic cost (setup/relocation penalties, not travel time) among goal-reaching plans; supervisor checks remain separate. Table 10.1 summarises the principal cases.

In the table, `validated_state` marks contract-valid facts. The field `active_goal_side` names the grounded rail. *Normal route* is the exterior-loop switch arrangement; a *topology block* is a segment-level, non-slot location; *clearance mode* temporarily relocates blockers; *capacity-safe* respects holding/separation limits; and a device group is one route's configured switches/stoppers.

Table 10.1: Principal symbolic decision cases in the current Room 315 PDDL domain.

Current symbolic situation	Decisive conditions	Applicable action and symbolic result
Canonical exterior-route preparation	Validated active side, connected stations, normal-route and named device-group facts	`prepare_switches`, then `open_stoppers`, establish `switches_ready`, `stoppers_open` and `path_ready` when those facts are still required
Direct move from an exact slot	Source occupancy agrees with the selected shuttle; target is `slot_free`; route is clear and ready; no clearance is pending	`move_shuttle_to_slot` releases the source occupancy and marks the target occupied and reserved
Switch-specific or segment-origin route	A topology block is known; a route to the free target is available and clear; no clearance is pending	`prepare_topology_route` or `prepare_slot_topology_route`, followed by the matching topology move, configures a legal route without inventing a source slot
Another shuttle blocks the required route	An ordered clearance is pending; the next relocation or staging destination is capacity-safe and its interior entry is clear	The clearance family enters `clearance_mode` and selects one permitted next relocation. Each completed step produces a fresh problem; finish or pause actions control route restoration
Mixed route requiring normalisation	Route reconfiguration is allowed; clearance mode is inactive; no relocation remains	`restore_normal_route` restores the exterior route and its readiness facts
Arrival or inspection is ready to close	Target station/slot and `DISABLED` controller-state facts hold, or a confirmed inspection is required	`stop_shuttle` and the terminal actions establish task completion; `inspect_state` completes an inspection without an actuation command

PDDL effects are predictions; the next cycle always uses the observed post-action state. The executive calls the POPF-backed PlanSys2 planner service (not its Executor), validates the plan and selects its first supported `PddlPlanStep/TranslatedPlanStep`: an action the translator can map to a checked command tied to the planning state.

10.5 Step 4—translate, supervise and move

The translator makes a sparse named `TranslatedPlanStep`; the supervisor applies chapter 8 before publishing a typed device/shuttle command, after which the kinematic controller updates shuttle state and Gazebo pose.

10.6 Step 5—check the result and decide what follows

The executive checks postconditions against newer state and, if incomplete, replans from it. Final-slot success requires the DZI/controller conditions in chapter 8.

10.7 Case B03 from instruction to arrival

For *Move the green shuttle to slot 4 on the right rail*, the campaign fixed request, goal and launch state but left planning open. From a cold start (fresh Gazebo process with no retained task, observation or controller state), the planner chose one direct step to R3's slot 4/DZI4R destination (table 10.2).

Table 10.2: English-to-motion record for campaign case B03.

Stage	Recorded result
Interactive request	*Move the green shuttle to slot 4 on the right rail.* The harness supplied `yes` after the rendered interpretation matched the case declaration
Confirmed task	Transport `room315_right_shuttle_3` on the right rail to slot 4
Current state	Visual state supplies current location and occupancy; a fresh PDDL problem is sent to POPF and the executive selects the first supported step
Execution step	The recorded initial state places R3 at slot 3 with slot 4 free. The motion portion contains one `move_shuttle_to_slot` step; it requires neither blocker clearance nor a topology relocation
Motion command	`move_shuttle_to_slot` for R3 from right slot 3 to right slot 4 within the Stäubli TX2-60L section at 0.2 m/s
Arrival	Two separately timestamped DZI4R readings report identity R3; the matching `ShuttleState` reports target slot 4 and `DISABLED`, and a newer visual observation records the final scene
Outcome	Every recorded postcondition is satisfied; the final status is `succeeded` with reason `task_goal_satisfied`

Other identity, colour and payload policies use the same workflow, grounding from the current scene before PDDL construction. Results are in chapter 11.

Chapter 11

Experiments and Results

11.1 Closed-loop campaign result

11.1.1 Campaign design and acceptance criteria

The predefined twelve-case matrix fixed each case ID, instruction, expected grounded goal and initial shuttle configuration. Slot goals named a specific destination, whereas station goals allowed any admissible slot. The environment record binds software, language setup, epoch-11 visual checkpoint, qualification/runtime manifests and PDDL domain. Before/after the campaign the runner rehashed this set and checked source/install parity (source equals built ROS overlay); each case rehashed the checkpoint, and the gateway verified the immutable qualification record. *Immutable* means hash-bound: edits invalidate its declared Gazebo-only component scope, which does not authorise an active service or physical use.

Every row used a fresh Gazebo process and case-specific ROS Domain ID, retaining no prior task, observation or controller state. The runner awaited the world, scene, controllers, cameras, authorised perception, planner, supervisor and gateway. Six rows per rail covered all eight identities and five exclusive families: colour alias, explicit identity, station, payload-qualified nearest and blocker-clearance (table 11.1).

The runner compared each displayed interpretation with its expected goal and entered interactive `yes` (never auto-confirm). Every observation required the configured schema and checkpoint. During campaign validation, each present shuttle in the initial and final observations also had to match its expected side, segment and payload state, with an absolute normalized along-segment position error no greater than 0.12. Success required the terminal state, all postconditions and supervisor checks, agreeing final DZI/stopped-controller readings, and clean camera/ROS-bag closure (a bag is a timestamped ROS-message recording).

Table 11.1: Closed-loop campaign rows and recorded terminal results.

Case	Case family	Rail / shuttle	Recorded task path	Steps / postconditions	Status
B01	Colour alias	Right / R1	Slot 1 → slot 2	1/1	Passed
B02	Explicit identity	Right / R2	Slot 2 → slot 3	1/1	Passed
B03	Colour alias	Right / R3	Slot 3 → slot 4	1/1	Passed
B04	Station target	Right / R4	Slot 4 → Yaskawa, slot 1	1/1	Passed
B05	Colour alias	Left / L1	Slot 1 → slot 2	1/1	Passed
B06	Explicit identity	Left / L2	Slot 2 → slot 3	1/1	Passed
B07	Colour alias	Left / L3	Slot 3 → slot 4	1/1	Passed
B08	Station target	Left / L4	Slot 4 → Yaskawa, slot 1	1/1	Passed
P01	Payload-qualified nearest	Right / R1	Loaded R1 selected; R2 cleared; slot 1 → slot 3	4/4	Passed
P02	Payload-qualified nearest	Left / L1	Loaded L1 selected; L2 cleared; slot 1 → slot 3	4/4	Passed
M01	Multi-shuttle route	Right / R1	R2 cleared; slot 1 → slot 3	4/4	Passed
M02	Multi-shuttle route	Left / L1	L2 cleared; slot 1 → slot 3	4/4	Passed

The twelve rows represent supported normal-workflow tasks in valid initial scenes. The automated suites in section 11.3 separately exercise stale state, sensor loss, emergency stop and protected-switch rejection.

11.1.2 Recorded result and scope

All 12 planned runs succeeded, and all 24 recorded step postconditions were satisfied. These 24 symbolic steps produced 48 low-level action proposals; the supervisor accepted all 48 and rejected none. Across the twelve runs, the executive made 24 plan attempts and recorded four occupancy-triggered replan events; it recorded no unknown-result retries and issued no safe aborts. All tasks ended with `status=succeeded` and reason `task_goal_satisfied`; all final effects were verified and every controller was stopped. The twelve bags contain 1,784 schema-compliant observations. Together, the runs trace the complete path from twelve distinct English instructions, through visual perception and fresh PDDL problem construction, to the expected shuttle movements in the selected Gazebo scenes.

A plan attempt is one planner-service request; an occupancy-triggered replan is a new request after observed occupancy changed. An unknown-result retry repeats an operation whose effect could not be established, whereas a safe abort ends the task without claiming success. The 1,784 observations are counted ROS messages carrying the active schema and checkpoint identity, not 1,784 independent scenes or additional accuracy trials.

The campaign summary records that physical deployment is disabled. These runs therefore provide positive-case Gazebo qualification evidence; they do not evaluate physical robots, reliability over repeated runs or fault handling. The corresponding record is indexed in appendix A.

Figure 11.1 compares the initial and final camera frames from case B03 and shows the corresponding change in the simulated scene.

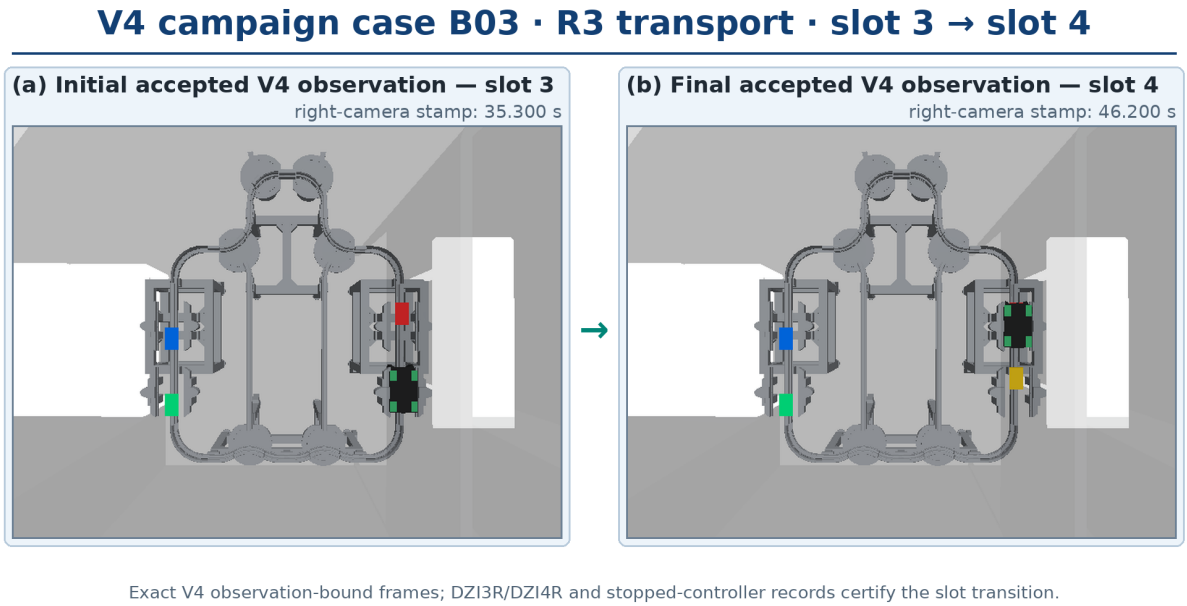


Figure 11.1: Exact right-camera frames selected by the initial and final accepted observations in the B03 ROS bag. The frames show the scene change; identity-bearing DZI3R/DZI4R and controller records establish R3's arrival at slot 4 and the final `DISABLED` controller state.

11.2 Final Test and runtime qualification

The Final Test evaluates figure 9.2. Its 168 learned values exclude topology-derived side, segment length and metric position; the diagnostic 200-value legacy adapter only reformats V4 and cannot enter control.

Dataset names retain the roles defined in chapter 9: Validation is the current 512-scene hard-case set, while the 30 July 2026 predecessor Test is consumed/denylisted and is not this Final Test.

11.2.1 One-shot Final Test

Frozen epoch 11 ran once under the protocol in chapter 9 on the frozen Final Test of 1,040 scenes, 2,080 images and 4,680 visible targets. Coverage used IDs/support counts without predictions, pixels or labels; no training, selection or calibration refit occurred. The run stayed within its partition and passed 76 base plus four runtime-threshold gates. Evidence is at `report/evidence/room 315_visual_v4_submission_2026-08-11/final_test`. The selected checkpoint, sealed images/labels and raw positive, fail-closed and post-promotion-smoke bags are published in the full-evidence release indexed in appendix A; they are not stored as Git blobs.

A gate is one predeclared Boolean acceptance check. The 76 base gates cover model quality and support across the global set, rails, segments, target zones, position bins and scene strata, plus the camera counterfactuals. The four runtime-threshold gates check minimum segment-confidence coverage, loaded-state coverage, joint-confidence coverage and selective segment accuracy. "Base" therefore denotes the common V4 acceptance suite, not the 1,024-scene base plan; the 80 gates are checks on the same evaluation, not 80 independent trials.

As defined in chapter 9, audited disjointness covers scene IDs, canonical generation-payload SHA-256 fingerprints, image bytes and complete label rows, not recurring presence configurations.

All classification metrics below use the 4,680 visible-shuttle targets defined in chapter 9. Top-1/top-2 accuracy asks whether the labelled public segment is the highest-scoring/one of the two highest-scoring classes. Supported-segment macro F1 averages per-segment F1 only over classes represented in the labels. A rail–segment cell is one side-by-segment stratum. The joint metric requires both the correct segment and the stated absolute normalised- coordinate tolerance. Along-segment error is evaluated offline against labelled ground truth and expressed as a fraction of segment length. The 0.12 threshold is used for planning-level evaluation, while 0.05 is a stricter diagnostic of localisation accuracy. Neither is an online runtime decision threshold. Correct-segment s_m errors are conditional on a correct segment; p95 is their 95th percentile. Bounding boxes use normalised top-left x, y, width and height, and IoU measures predicted/labelled overlap. Boundary and switch zones are the preregistered target strata near segment boundaries and controlled switch regions, respectively; their exact bounds remain in the evaluation controls.

Table 11.2: Principal metrics from the one-shot Final Test.

Metric	Global	Left rail	Right rail
Public-segment top-1 accuracy	99.9359%	99.8726%	100.0000%
Public-segment top-2 accuracy	100.0000%	100.0000%	100.0000%
Supported-segment macro F1	99.9117%	99.7460%	100.0000%
Loaded-state accuracy	99.9145%	99.9151%	99.9140%
Joint segment and $\lvert\Delta s_{ratio}\rvert \leq 0.05$	90.8974%	89.8089%	92.0000%
Joint segment and $\lvert\Delta s_{ratio}\rvert \leq 0.12$	99.0385%	98.9384%	99.1398%
Correct-segment s_m MAE	0.01573 m	0.01552 m	0.01594 m
Correct-segment s_m 95th percentile	0.05850 m	0.05660 m	0.06030 m
Mean bounding-box IoU	0.83400	0.83103	0.83701

All 28 supported rail–segment cells had observations and no supported segment had zero recall. The worst cell was left–A14, with 94.6429% recall on 56 targets. Boundary-zone segment accuracy was 99.1304% on 230 targets and the switch-zone result was 100% on 134 targets. These stratified results are more informative for planning than a global average alone.

At the frozen segment-confidence threshold, 4,678/4,680 targets were retained: 99.9573% coverage and 99.9572% selective segment accuracy, so two targets abstained. Loaded-state coverage was 4,680/4,680 with 99.9145% selective accuracy; joint confidence coverage was 4,678/4,680. In the camera counterfactuals, blanking or shuffling the opposite camera changed the own-side outputs by zero at the recorded precision. Swapping camera order reduced segment top-1 accuracy from 99.9359% to 7.7991%, a 92.1368-point drop, and reduced the joint 0.05 result from 90.8974% to 0.8120%. This confirms the enforced camera order and supports the rail-isolation contract.

Here, coverage is the fraction whose score meets the frozen floor, selective accuracy is accuracy among retained targets, and abstention means not accepting a target estimate below that floor. Joint coverage requires both segment and loaded-state score checks. Because the runtime uses an all-present-targets policy, one required target's abstention rejects the complete paired observation. The counterfactuals deliberately blank or cross-scene shuffle the opposite camera, or swap the declared camera order, and compare the resulting outputs at the evaluator's stored numeric precision.

The Final Test is the principal visual-model result in this report. It contains generated Gazebo scenes with fixed simulated cameras and the known Room 315 fleet. Its scope is visual evaluation in simulation: it does not assess physical cameras or control, and it does not enable planner mutation or actuation.

11.2.2 Development evidence

The checkpoint passed 76/76 Validation and 74/74 Canary gates. Validation drove selection, patience, calibration and the score floor; the Canary only checked a frozen candidate for degradation and could block manual Gazebo promotion, never alter the model/calibration. Neither is Final Test evidence (table 11.3).

Table 11.3: Validation and regression metrics supporting the Final Test.

Metric	Validation	Exposed development Canary
Public-segment top-1 accuracy	99.9113%	100.0000%
Public-segment macro F1	99.8799%	100.0000%
Worst side–segment-cell recall	97.6190%	100.0000%
Loaded-state accuracy	99.8669%	100.0000%
Joint segment and $\lvert\Delta s_{ratio}\rvert \leq 0.05$	94.9867%	94.5622%
Correct-segment s_m MAE	0.01867 m	0.02015 m
Correct-segment s_m 95th percentile	0.06483 m	0.07002 m
Mean bounding-box IoU	0.83427	0.83466
Acceptance gates	76/76	74/74

The calibrated segment-confidence threshold was fitted on Validation only. Loaded confidence remains an uncalibrated softmax decision value, and the online calibrated segment-score floor is 0.9139089813389109, while the online binary loaded-score floor is 0.5. These score floors are distinct from the offline coordinate-error criteria defined above. The runtime rejects a complete frame when any identity declared present fails its required input, confidence or contract check; identities declared absent contribute no accepted visual facts.

11.2.3 Observation-only qualification evidence

Table 11.4 reports observation-only checks; 169 identity checks exceed frame count because frames can contain several targets. All eight injected faults—wrong manifest/checkpoint/topology hashes, reversed cameras, stale left/right images, stale presence and unknown identity—were rejected.

The observation-only manifest kept fusion dry-run, PlanSys2 updates disabled and task execution/actuation absent; its records contain no control action or physical approval (appendix A).

A *shadow* consumes the same frames on isolated, non-actuating topics; a *smoke* is a small sanity check; a *reobservation* repeats inference on the declared scene.

Table 11.4: Observation-only qualification evidence and execution scope.

Check	Recorded result	Control scope
Same-frame shadow	55 paired frames; 52 accepted (94.5455%); no errors or wrong-side identities	Isolated shadow topics; zero PlanSys2 mutations and zero actuation commands
Seven-scene acceptance	7/7 complete; 53/53 reobservations met the planning criterion (correct segment and $\lvert\Delta s_{ratio}\rvert \leq 0.12$); 150/169 identity checks met the stricter $\lvert\Delta s_{ratio}\rvert \leq 0.05$ diagnostic criterion	Observation only; scenario ground truth excluded from model input
Visual-input fault injection	8/8 specified faults rejected under the fail-closed policy	No plan client, PlanSys2 update or actuation command
Dense-scene dry-run	Immutable manifest review passed; 10/10 matching reobservations	Gazebo dry-run only; PlanSys2 and task execution disabled; zero actuation

11.2.4 Closed-loop fault and runtime qualification

A separate campaign injected five faults in fresh Gazebo processes and isolated ROS domains. F01 and F02 tested corrupt authorisation and an invalid runtime manifest; both were rejected before motion. F03 removed the planner service, F04 triggered an emergency stop during a deliberate long move, and F05 removed right-sensor feedback during a long move. All five met their safety criteria: no faulted run falsely reported completion, controllers ended `DISABLED`, and all 424 recorded observations complied with the active schema. None of these results authorises physical deployment.

The approved V4 runtime is restricted to Gazebo and starts with command execution disabled. Manual approval selects the hash-bound runtime; the operator then enables execution for the session and confirms the interpreted task. Planning and execution may then proceed automatically, while every low-level command remains subject to supervisor authorisation.

A targeted B01 smoke test revalidated the approved manifest in a controlled Gazebo run. The case completed its actuating step and postcondition, reached the expected terminal state, and stopped the controller correctly; all 88 observations matched the active schema and checkpoint. This single-case check complements the preceding 12-run campaign. The approval record binds the qualification manifest, positive-campaign summary and fault bundle. These records and the B01 evidence are indexed with their checksums in appendix A.

11.3 Automated software checks

Before the complete runs, automated tests checked the software used by the workflow. Static and unit checks cover geometry, configuration and schema rules. Component and controlled-integration tests exercise providers, task construction, planning, translation, supervision and effect checking in controlled harnesses. They also include fault and protective cases. The integrated campaign then evaluates the normal-path components together in the Gazebo workflow.

On 7 August, three named suites were executed. The focused contract and executive suite passed 190/190 checks in 4.10 s, including the Torch-dependent visual-runtime case through the campaign's visual Python path. A transport suite passed 96/96 checks in 1.48 s, covering kinematic paths, topology, headway, reservations, fleet supervision, devices, identity and static shuttle clearance. The complementary operator-capability suite passed 42/42 checks in 0.41 s, covering the robot command helper, launch propagation, rail devices, marker/sensor lifecycle and switch-feedback rules. The suites overlap in a small number of files and are therefore reported separately rather than summed. Full commands and results are described in appendix A.

11.3.1 Language-interface evaluation

The nine non-control cases in the unchanged ten-case English corpus were each executed once through the configured local semantic gateway, strict envelope parser and field fusion; successfully parsed drafts were then passed to domain validation. The dialogue manager handled the cancellation case before model inference. All ten declared final outcomes matched: four supported goals produced the expected constraints, three incomplete or ambiguous requests required clarification, two inconsistent or invalid requests were rejected, and the cancellation command produced no task. All five incomplete, ambiguous, inconsistent or invalid requests ended in clarification or rejection.

The configured local backend was invoked exactly once in all nine non-control cases. Eight outputs passed the strict semantic-envelope schema without fallback. For the invalid-slot case, the model output itself contained slot 9; the schema rejected that envelope and the request remained an error. The cancellation command bypassed inference. This ten-utterance study evaluates the implemented Room 315 English contract. The retained case records and configuration are described in appendix A.

Here, the cancellation row is the deterministic control case; the other nine invoke the model. "Without fallback" means that no parser repair or substitute semantic result was used after those eight schema-valid model envelopes.

11.3.2 All-robot launch check

A separate cold-start check launched the full-floor profile with `robots:=all` in an isolated ROS domain and Gazebo partition. Here, Gazebo partition means a Gazebo Transport communication namespace, not a visual-data partition. All six configured robot models were present; each exposed its namespaced joint command and feedback topics and produced a fresh `JointState` sample, while both TIAGo variants exposed independent base-velocity topics. The active log before planned shutdown was free of the declared error patterns. This check covers launch composition, namespace separation and feedback availability. Motion, simultaneous operation and repeatability require dedicated runs.

Figure 11.2 orders the evaluation layers by system integration. Complete records are indexed in appendix A; operator tools and procedures are documented in `runbook.html` and `docs/QUICK_START_AND_FEATURE_GUIDE.md`.

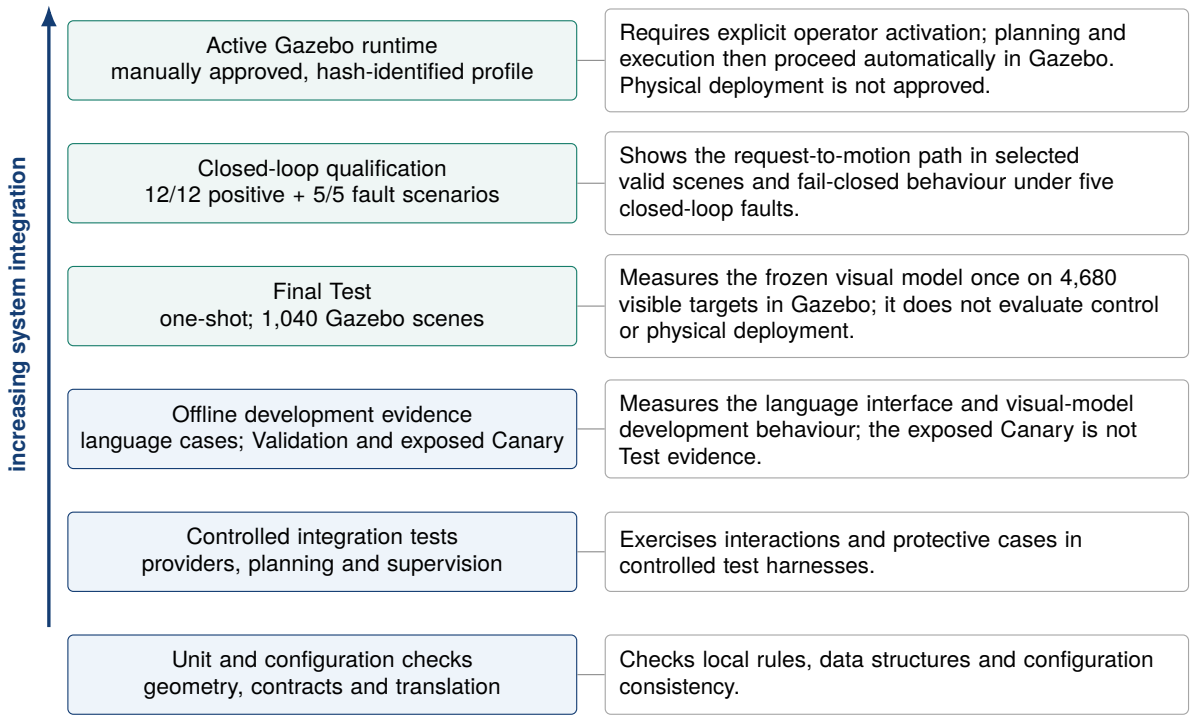


Figure 11.2: Layers are ordered by increasing system integration; each addresses a distinct evaluation scope.

11.4 Review of the internship objectives

Table 11.5 returns to the O1–O4 objectives introduced in chapter 1 and summarises how each one was assessed.

Table 11.5: Assessment of the four internship objectives.

Objective	Main result	How it was evaluated
O1—simulation platform	Third-floor and Room 315 Gazebo models plus one all-robot cold-start check: six of six configured models, namespaced command/feedback topics and fresh state samples	One full-floor cold-start launch and interface check
O2—multi-shuttle transport	Calibrated Hermite rail geometry, explicit switch-dependent topology, eight stable shuttle identities, typed devices, reservations, headway checks and limited recovery on both rails; 96/96 named transport checks and route-clearance cases in the integrated campaign	Named software tests and declared simulated campaign scenes
O3—contracts and supervision	Versioned command, state and effect contracts; sparse device updates; freshness and consistency gates; supervisor rejection cases; and post-step effect checks covered by the 190/190 focused checks and the integrated runs	Named software tests and simulated campaign records
O4—neuro-symbolic workflow	Rail-isolated checkpoint with a passed one-shot Final Test; closed-loop qualification with 12/12 positive Gazebo runs and 1,784 observations; 5/5 closed-loop fault cases with 424 observations, no false success and disabled controllers; manually approved Gazebo runtime	Ten language cases, 1,040 Final Test scenes with 4,680 visible targets, twelve positive campaign scenes, five fault scenes and observation-only qualification checks

Chapter 12

Limitations and Perspectives

12.1 Limitations

The first phase integrated all six configured robots, namespaced commands and feedback, but its one full-floor cold start does not establish simultaneous- motion accuracy or repeatability. Individual motion exists; collision-aware, contact-verified autonomous manipulation does not.

The integrated campaigns are Gazebo-only. Their kinematic shuttle backend supports topology and task-execution studies but omits torque, slip, braking, play and payload inertia; its software supervisor is not a certified safety function.

The perception study uses the known shuttle fleet, fixed simulated cameras and generated scene families. Its one-shot Final Test is procedurally separate and disjoint from the current training, hard-case Validation and Canary inputs at the audited exact scene-ID, canonical generation-parameter fingerprint, image-byte and complete-label-row levels. This does not imply presence-configuration or perception-distribution independence: presence configurations intentionally recur, and the Test still characterises the same camera geometry and synthetic distribution rather than unseen hardware, lighting or installation conditions. Identity presence comes from an external registry and rail side from the L/R identity contract; these are assumptions, not learned achievements. Segment confidence is Validation-calibrated, while the loaded-state softmax value is not a calibrated probability. The 0.05 and 0.12 coordinate-error criteria use ground-truth labels offline and are not confidence gates that the live runtime can evaluate.

A post-hoc registry audit found support overlap below the sample level. Hard Training and Hard-case Validation share 23 complete visual-target payloads, plus 14 trajectory and 14 geometry fingerprints; Hard Training and Canary share 14 target payloads, plus eight trajectory and eight geometry fingerprints. This is target/state support overlap, not sample or image reuse: scene IDs, images, complete label rows, scene families and generated variants remain disjoint.

The twelve command-producing rows cover selected valid scenes; the Final Test is perception-only, and manual approval remains Gazebo-only.

Each positive row, fault and all-robot check ran once, so controlled-seed repetition (the same protocol under declared pseudorandom seeds), long-outage recovery and profile switching remain unmeasured. Shuttle arrival uses command-linked observations; device-report correlation, longer circular wait-for cycles and temporal non-progress remain open.

The CLI confirmation heuristic labels transport high, otherwise shuttle/ selection/rail targets medium, and all else low; these prompt labels are not a certified risk assessment. The subscriber trusts the ROS graph and official CLI, so multi-user service requires authenticated sources or graph isolation and sender identity.

The language study covers ten selected English requests, including clarification, rejection and cancellation. Open-vocabulary and multilingual evaluation remain future extensions.

12.2 Next evaluation steps

Table 12.1 orders the next simulation-focused tasks by the question each one would answer.

Table 12.1: Priority experiments for extending the evaluation.

Priority	Engineering task	Purpose
P0	Generate a separate visual-shift evaluation set	Measure transfer to varied camera poses, lighting, textures and sensor noise without tuning on the completed one-shot Final Test
P0	Repeat the seven-scene review across controlled seeds	Measure same-frame acceptance, exact segment/payload agreement and longitudinal-error variation in sparse, dense and blocker arrangements
P0	Inject and recover faults in the live ROS graph	Extend the completed eight-case fail-closed contract check with timed camera/presence loss, node restart, planner timeout and emergency-stop recovery
P0	Repeat the positive and fault campaigns across controlled seeds	Quantify completion, abstention and recovery variation while preserving the exact schema, checkpoint, terminal-effect and controller-stop checks
P1	Relate switch and stopper acknowledgements to commands	Correlate each device result with a matching state update newer than the command
P1	Mirror and periodically restore-test the reproducibility release	Verify the published archive and nested checksums from a fresh machine, retain an independent institutional copy and document any future evidence revision

12.3 Continued use at MFJA

For teaching, students can launch selected robots, inspect interfaces, alter a rail scene, generate PDDL and trace a supervised step from a configured cold start using the runbooks.

The same assets provide a reference benchmark. Alternative planners, perception checkpoints or task-grounding methods can use the fixed case matrix without changing controller behaviour or arrival checks. Model selection must use Validation; the Canary remains a regression check and the Final Test remains the fixed one-shot evaluation. Added camera, occlusion, clutter or payload perturbations belong in a separate domain-shift partition. Here, domain shift means a change in the visual data distribution, not a change to the PDDL action domain or ROS Domain ID.

12.4 Focused research extensions

Planner/runtime changes must retain canonical rail alignment, executable translation/effect checks, schema/checkpoint allowlists, presence-aware abstention, calibrated confidence and operator approval. Only independently tested atomic (non-overlapping) tasks should progress to concurrency, after reservation and deadlock evaluation.

Chapter 13

Conclusion

The four internship objectives were completed in simulation. The project delivers a maintainable ROS 2 Jazzy and Gazebo Harmonic environment with selective launch, namespaced command and feedback interfaces, and verified motion of the industrial and mobile robots. For Room 315 it adds geometrically calibrated rail paths, switch-dependent routes, typed contracts, planning and supervision, English task grounding and rail-isolated paired-camera perception. Table 11.5 maps each objective to its evaluation evidence.

The language interface produced the intended outcome in all ten declared cases, including clarification, rejection and cancellation. The complete request-to-motion path succeeded in all 12 isolated cold-start runs; every final effect was verified and every controller stopped. The postcondition, supervision, planning, replanning and observation records are reported in section 11.1.

The independent one-shot Final Test evaluated 4,680 visible shuttle instances across 1,040 Gazebo scenes. Public-segment top-1 accuracy reached 99.9359%, while joint segment and longitudinal accuracy at $|\Delta s_{ratio}| \leq 0.05$ reached 90.8974%; all 80 predefined gates passed. Table 11.2 reports the remaining per-rail, loaded-state, localisation and bounding-box metrics. The camera counterfactuals support the intended rail isolation and enforced camera order.

The observation-only checks ran in dry-run mode without issuing control actions. The separate closed-loop fault campaign passed 5/5 declared scenarios with no false success and stopped controllers. The approved runtime profile is limited to `gazebo_v4_closed_loop_runtime_only`. Command execution starts disabled and requires explicit operator activation; once enabled and given a confirmed task, closed-loop planning and execution proceed automatically in Gazebo. Physical deployment is not approved. The task gateway, rather than the inference node, owns planning-state construction and Gazebo actuation.

For MFJA, the result is a reusable platform for practical teaching, software integration, synthetic-data generation, and planning or perception studies. For me, the central lesson was how to divide a broad integration request into small, testable components and connect them through explicit contracts. I also prepared configuration, regression tests and run instructions so that the host team can continue and extend the work.

The evaluation priorities are listed in Table 12.1. Within the present scope, command authority remains limited to Gazebo; any future transfer to the physical MFJA platforms should preserve the same explicit validation, supervision and effect-verification boundaries.

Bibliography

[1] AIP-PRIMECA Occitanie. MFJA 3rd floor Gazebo simulation. https://github.com/aip-primeca-occitanie/mfja_3rd_floor_gz, 2026. Project repository, accessed 6 August 2026.

[2] Amanda Jane Coles, Andrew I. Coles, Maria Fox, and Derek Long. Forward-chaining partial-order planning. In *Proceedings of the Twentieth International Conference on Automated Planning and Scheduling*, pages 42–49, 2010. doi: 10.1609/icaps.v20i1.13403.

[3] Timothy Duggan, Pierrick Lorang, Hong Lu, and Matthias Scheutz. The price is not right: Neuro-symbolic methods outperform VLAs on structured long-horizon manipulation tasks with significantly lower energy consumption. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, 2026. URL https://hrilab.tufts.edu/publications/dugganetal26icra.pdf.

[4] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. Software repository, 2023. URL https://github.com/ggml-org/llama.cpp. Accessed 7 August 2026.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.

[6] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, et al. Do As I Can, Not As I Say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2023. URL https://proceedings.mlr.press/v205/ichter23a.html.

[7] INSEE. NAF rev. 2, subclass 85.42Z: Higher education. https://www.insee.fr/fr/metadonnees/nafr2/sousClasse/85.42z, 2025. Accessed 4 August 2026.

[8] INSEE. Avis de situation au répertoire SIRENE: Université de Toulouse, SIRET 938 271 392 00012. https://api-avis-situation-sirene.insee.fr/identification/pdf/93827139200012, 2026. Official SIRENE record; accessed 11 August 2026.

[9] International Organization for Standardization. ISO 23247-1:2021 automation systems and integration—digital twin framework for manufacturing—part 1: Overview and general principles, October 2021. URL https://www.iso.org/standard/75066.html.

[10] Nishanth Kumar, William Shen, Fabio Ramos, Dieter Fox, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Caelan Reed Garrett. Open-world task and motion planning via vision-language model generated constraints. *IEEE Robotics and Automation Letters*, 11(3):3366–3373, 2026. doi: 10.1109/LRA.2026.3656799. URL https://arxiv.org/abs/2411.08253.

[11] Francisco Martín, Jonatan Ginés Clavero, Vicente Matellán, and Francisco J. Rodríguez. PlanSys2: A planning system framework for ROS2. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9742–9749, 2021. doi: 10.1109/IROS51168.2021.9636544.

[12] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL—the planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, October 1998. URL https://ipc08.icaps-conference.org/deterministic/data/mcdermott-et-al-tr-1998.pdf. Version 1.2.

[13] Open Robotics. Getting started with Gazebo—Gazebo Harmonic documentation. https://gazebosim.org/docs/harmonic/getstarted/, 2026. Accessed 4 August 2026.

[14] Open Robotics. Use ROS 2 to interact with Gazebo. https://gazebosim.org/docs/harmonic/ros2_integration/, 2026. Accessed 4 August 2026.

[15] Open Robotics. Interfaces (topics, services, actions)—ROS 2 Jazzy documentation. https://docs.ros.org/en/jazzy/Concepts/Basic/Interfaces-Topics-Services-Actions.html, 2026. Accessed 5 August 2026.

[16] Open Robotics. Quality of service settings—ROS 2 Jazzy documentation. https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html, 2026. Accessed 4 August 2026.

[17] Open Source Robotics Foundation. SDFormat 1.12 specification: Root and sensor elements. https://sdformat.org/spec/1.12/ and https://sdformat.org/spec/1.12/sensor/, 2026. Version 1.12; accessed 12 August 2026.

[18] Qwen Team. Qwen2.5-1.5B-Instruct-GGUF model card. Hugging Face model repository, 2024. URL https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF. Official model card documenting the GGUF distribution and Q4_K_M quantisation; accessed 11 August 2026.

[19] Université de Toulouse. La Maison de la Formation Jacqueline Auriol (MFJA). https://www.univ-tlse3.fr/lieux-de-ressources/la-maison-de-la-formation-jacqueline-auriol-mfja, 2026. Accessed 4 August 2026.

[20] Université de Toulouse. Schéma de promotion des achats publics socialement et écologiquement responsables 2026–2029. https://www.univ-tlse3.fr/medias/fichier/2026-03-ca-083_1773392372102-pdf, 2026. Institutional figures on p. 5; definitive SPASER adopted by the university council on 9 March 2026; document dated 13 March 2026; accessed 12 August 2026.

[21] Université d'Évry Paris-Saclay. Évaluation de la formation en entreprise (Module Stage M2): Le rapport. Internal university guidance, 2026. Official internship-report instructions, June 2026, p. 17, Section V: use of generative artificial intelligence.

[22] Université d'Évry Val d'Essonne and Université de Toulouse. Convention de stage 2025–2026, 2026. Signed internal document, p. 1: host Université de Toulouse, assigned service MFJA, placement subject and entrusted activities; 2 March–28 August 2026, 26 weeks and 868 hours of effective presence.

[23] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, Yaxin Peng, Feifei Feng, and Jian Tang. TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 10(4): 3988–3995, 2025. doi: 10.1109/LRA.2025.3544909. URL https://arxiv.org/abs/2409.12514.

[24] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115. URL https://arxiv.org/abs/2412.15115.

[25] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.

Chapter A

Reproducibility Records

A.1 Source, environment and build

The project repository is identified by [1]. The reference environment is Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic and Python 3.12. The revision carrying this report stores the implementation, active runtime configurations, tests, lightweight evidence and submission PDF together. A clean checkout of that revision therefore reconstructs the source handover. The model checkpoint, sealed dataset and raw ROS bags are external objects identified by the fingerprints below rather than files stored in Git; their byte-exact current-V4 copies are published in the full-evidence release identified below. Here, a checkpoint is a saved network-weight state, a sealed dataset is a frozen hash-identified dataset (not an encrypted or still-unseen one), and a ROS bag is a timestamped recording of ROS messages. An external object is retained outside Git because of size, while its digest fixes the object used by the reported run.

From the workspace root, the source revision can be recorded and the installed overlay rebuilt with the commands below. In ROS/colcon terminology, an overlay is this workspace's build and install output layered over the base ROS installation.

```
source /opt/ros/jazzy/setup.bash
git -C src/mfja_3rd_floor_gz rev-parse HEAD
git -C src/mfja_3rd_floor_gz status --short
colcon build --symlink-install --base-paths src/mfja_3rd_floor_gz
source install/setup.bash
```

From `report/`, `make check` builds the A4 PDF, rejects unresolved references or citations and checks for overfull boxes. The retained build log and delivery-PDF fingerprint are stored under `report/evidence/`; keeping the PDF digest outside the PDF avoids a self-referential identity. Operating procedures and launch arguments are documented in `runbook.html`, `docs/INSTALLATION.md`, `docs/QUICK_START_AND_FEATURE_GUIDE.md` and `docs/ROOM315_LANGUAGE_TO_MOTION_RUNTIME.md`.

A.2 Primary evidence identity

The repository-relative evidence package is `report/evidence/room315_visual_v4_submission_2026-08-11`. Its `evidence_manifest.json` has SHA-256 `840787b617e0f671628dfc7d8122ff559d76664506ff8f94ac31d043737da446`, and the package-wide `SHA256SUMS` file has SHA-256 `dec74565a8ccfba57a32d83dfdd03236daeafeff226d5243594e328fa4adc5ab`. Together they cryptographically bind the lightweight copies and external-object fingerprints, and map each reported result to its source record. The complete per-file hashes and JSON Pointers remain in that package and in `report/evidence/ACTIVE_EVIDENCE.md`; they are not repeated in this appendix.

The corresponding full payload is published at the Room 315 Visual Model V4 full-evidence release. Its tag `room315-visual-v4-evidence-2026-08-11` targets revision `0d19e1601d57416b83c871c1a8d413ec0dd523a6`, which introduced the V4 runtime and the compact package; that package is byte-identical at the revision carrying this report. The archive is `247825706` bytes with SHA-256 `35b583baca4f45eed6aad659c253180d00f1a5830ce389266ba714a4445a8ecc`. Its 6,492 files include the selected checkpoint, the sealed 1,040-scene Test with 2,080 images, the final active runtime bundle and 18 MCAP recordings from the positive, fail-closed and post-promotion-

smoke runs. The release tag's V4 identifies the visual-model generation; it is unrelated to the historical repository tag `version-4`.

Lightweight means that the package retains manifests, controls, results and checksums but not the large checkpoint, images or raw bags. Binding means that any content change produces a different SHA-256 and fails verification; it does not make a file physically impossible to edit. A JSON Pointer is the recorded path to one field inside a JSON document.

The selected epoch-11 visual checkpoint has SHA-256 `869d64049b0092c37d21a4c8b910dc6b91954527e0e49c5694fa82dce570f40d`. The sealed 1,040-scene Final Test has dataset fingerprint `226f4b5889f7d8b66bccc87d3e8dd494f710c8f4b228fab4851d6da7448e2eef`. These fingerprints identify the two principal external objects used by the visual evaluation; each is the SHA-256 identifier recorded by its governing protocol. Some copied machine-generated records preserve absolute archival paths from the evaluation host. They are retained for provenance but are not submission locations; the authored evidence index uses relative paths.

The release assets can be downloaded and verified without adding their payload to Git:

```
TAG=room315-visual-v4-evidence-2026-08-11
gh release download "$TAG" \
  --repo aip-primeca-occitanie/mfja_3rd_floor_gz
sha256sum -c \
  room315_visual_v4_full_evidence_2026-08-11.tar.zst.sha256
tar --zstd -xf \
  room315_visual_v4_full_evidence_2026-08-11.tar.zst
cd room315_visual_v4_full_evidence_2026-08-11
sha256sum --strict -c SHA256SUMS
```

The repository landing record `report/evidence/ROOM315_VISUAL_V4_RELEASE.md` repeats the asset identity, scope and extraction instructions. Raw records preserve archival host paths and reviewer/name provenance byte-for-byte. They remain Gazebo-only evidence and confer no physical-deployment authority.

A.3 Result-to-record index

Table A.1 gives the shortest path from each reported result to its inspectable record. Paths beginning with `model/`, `final_test/` or `runtime/` are relative to the primary evidence package above.

Table A.1: Records supporting the reported results.

Result group	Records and scope
Visual model and Final Test	`model/` contains the effective training configuration, input fingerprints, selected-run report and hard-case Validation records; `canary/` is the labelled development regression check. `final_test/final/controls/` contains the sealed controls and one-shot evaluation contract, while `final_test/final/results/` contains the immutable completion ledger and measurements. The controls freeze what may run; the ledger records that the single permitted attempt was consumed and completed.

Result group	Records and scope
Runtime qualification	`runtime/shadow/`, `runtime/campaign/` and `runtime/safety/` contain the observation-only comparison, scene campaign and visual-input fault checks. `runtime/closed_loop/positive/`, `runtime/closed_loop/fault/` and `runtime/closed_loop/active_runtime/` contain the positive campaign, fault campaign, manually approved Gazebo runtime and its selected-case smoke. Exact per-record digests are available in `report/evidence/ACTIVE_EVIDENCE.md`.
Language interface	`report/evidence/room315_language_semantic_evaluation_v3` contains the pinned ten-case corpus, configuration, raw model envelopes, fusion traces, environment and checksums. The directory suffix is an evidence revision and does not select the visual runtime.
Software and full-floor checks	The recorded component commands and outputs are the three `pytest_current_source_*_2026-08-07.log` files under `report/evidence/`. The isolated all-robot launch/interface record is `report/evidence/multi_robot_cold_start_smoke_2026-08-07`.
Experimental camera figure	The exact B03 frames, extraction program, provenance and checksums for figure 11.1 are under `report/evidence/room315_v4_closed_loop_campaign_figures_2026-08-12`.
Active runtime configuration	The active files are `mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml` and `task_execution_runtime.yaml`. The task configuration defaults to `execution_enabled=false`.